PRINT EDITION

Building AI Agents with Claude

The mobile course, condensed for offline reading. Every module's big idea, analogy, mechanics, pseudocode, misconceptions, takeaway, and quiz — in one document.

32 modules · From Hello World to Autonomous Production Systems

MODULE

The Agent Lifecycle — See the Whole Picture First

Card 1 of 9
Track: Course Overview
MODULE 0 of 30

The Agent Lifecycle — See the Whole Picture First

Before you write a single line of code, understand what an AI agent really is, how it works, and where every module in this 30-module course fits. This is your map.

10 min read

Card 2 of 9

The Big Idea

An **AI agent** is a program that uses a large language model (LLM) as its brain, connects it to tools like databases, APIs, and files, and runs in a loop — thinking, acting, observing results, and repeating until the task is complete.

Unlike a chatbot that gives one answer from memory and stops, an agent **keeps working until the job is done**. It can look things up, make decisions based on what it finds, and take multiple steps to reach a complete answer.

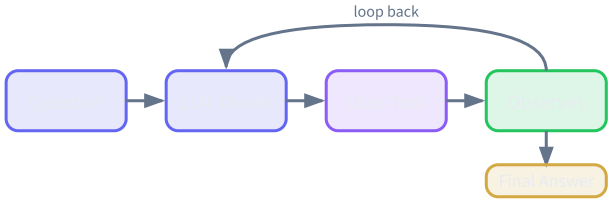
Three capabilities make this possible:

1. **An LLM as its brain** — it reads input, reasons about what to do, and decides the next action
2. **Tools as its hands** — it can call APIs, query databases, and read files
3. **A loop as its engine** — it keeps thinking and acting until the task is complete

Card 3 of 9

The Analogy

> Airport Information Desk vs. Travel Assistant
> **Before (chatbot):** You walk up to an airport information desk and ask “When does the next flight to Chicago leave?” The person answers from memory: “I think it is around 3pm.” Maybe right, maybe wrong — they cannot check.
>
> **The pain:** What if the answer is wrong? What if there are cheaper flights, or you have frequent flyer miles? The information desk can only tell you what they remember. They cannot look anything up, compare options, or book a ticket.
>
> **The agent version:** A travel assistant hears your question, pulls up the flight database, checks 4 options, compares prices, notices your frequent flyer miles, applies them, and books the best option. Same question — but the assistant *used tools*, *made decisions*, and *kept working* until the task was complete.



Card 4 of 9

The 7 Building Blocks

Every production agent has these 7 components. Remove any one and the agent is incomplete.

1. Brain (LLM) Reads input, reasons, decides next action

M01–M04

2. Tools Calls APIs, queries databases, reads files

M05–M07

3. Memory Conversation history, document retrieval / RAG

M08–M11

4. Plan Breaks complex tasks into subtasks

M12–M15

5. Guardrails Validates inputs/outputs, prevents harm

M16–M18

6. Eyes (Observability) Logs decisions, traces tool calls, monitors

M19–M20

7. Home (Deployment) Production deployment, scaling, cost control

M21–M22

Card 5 of 9

The Universal Agent Pattern

Every agent — from a simple lookup tool to an enterprise system — is a variation of this pattern:

```
// The universal agent loop
USER asks a question
messages = [user's question]

WHILE true:
  response = SEND messages to Claude

  IF response contains tool_use:
    tool_name = response.tool_name
    tool_input = response.tool_input
    result = RUN tool_name(tool_input)
    APPEND result to messages
  ELSE:
    RETURN response to user
    BREAK
```

That is it. The LLM decides whether to use a tool or give a final answer. Your code runs the tool and feeds the result back. The loop continues until the LLM has enough information to respond. The rest of this course teaches every variation, optimization, and production concern built on top of this pattern.

Card 6 of 9

Common Misconceptions

Wrong mental models that trip up beginners:

"Agents are autonomous AI that run on their own"

No. An agent runs when your code calls it and stops when the task is done. It is your code with an LLM inside a loop — not a self-directed entity. It does not think, plan, or act between calls.

"An agent is just a smarter chatbot"

The difference is not intelligence — it is the ability to *use tools*, *make decisions*, and *iterate until done*. A chatbot gives one response from memory. An agent takes multiple steps to complete a task.

"Agents must be complex — hundreds of lines"

The core pattern is roughly 10 lines: a while loop that calls an LLM, checks for tool use, runs the tool, and repeats. Everything else — memory, planning, guardrails — is added incrementally.

Card 7 of 9

Key Takeaway

An Agent = LLM + Tools + Loop

The LLM is the brain, tools are the hands, and the loop keeps it working until the job is done.

Production agents need all 7 building blocks across 5 lifecycle stages: **Design** → **Build** → **Protect** → **Observe** → **Deploy**.

Most tutorials stop at Build. This course covers all five stages across 30 modules, 5 capstone projects, and a certification prep track.

The LLM Mental Model

Understand what a Large Language Model really does, how it processes your text, and why this mental model changes everything you build.

The World's Most Well-Read Autocomplete

A Large Language Model like Claude is the world's most well-read autocomplete. It doesn't "understand" language the way humans do — it predicts the most likely next token based on patterns learned from billions of documents.

Every impressive thing an LLM does — writing code, answering questions, translating — is a **side effect** of getting extremely good at next-token prediction.

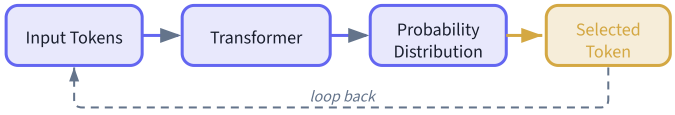
This mental model changes how you write prompts, design tools, and build guardrails. Once you see Claude as a prediction engine rather than a knowledge database, every design decision in this course will make more sense.

Autocomplete That Went to Every University

Your phone's autocomplete has read your messages; Claude has read billions of documents — books, code, scientific papers — and uses all of that to predict what comes next.

Instead of following hand-written rules, it learned patterns from a mountain of text, so it can handle questions nobody explicitly programmed it for.

It's **autocomplete that went to every university, read every manual, and practiced every writing style.**



Five Steps of Text Generation

1. **TOKENIZE** — Your text is split into tokens (subword chunks). "The capital of France is" becomes 5 tokens.
2. **PROCESS** — All tokens flow through transformer layers simultaneously (reads all at once).
3. **PREDICT** — The model outputs a probability for every possible next token: "Paris" 92%, "the" 3%, "located" 2%...
4. **SELECT** — One token is chosen based on temperature/sampling settings.
5. **REPEAT** — The selected token feeds back as input. Generate the next token. Repeat until done.

Key insight: Claude reads ALL input at once but writes output ONE token at a time.

The Generation Loop

Here is the entire mechanism behind every LLM response, distilled into a few lines:

```
INPUT text = "The capital of France is"
tokens = TOKENIZE(text)  // split into subword pieces

REPEAT until stop condition:
  probabilities = TRANSFORMER(tokens)  // process all at once
  next_token = SAMPLE(probabilities, temperature)
  APPEND next_token to tokens

  IF next_token == STOP_TOKEN:
    BREAK

OUTPUT = DETOKENIZE(tokens)  // convert back to text
```

The entire loop — tokenize, predict probabilities, sample one token, append, repeat — is how Claude generates every single response. The "intelligence" lives in the TRANSFORMER step, where billions of learned parameters assign probabilities.

What People Get Wrong

"LLMs understand language like humans do"

No. They find statistical patterns at massive scale. The results look like understanding, but the mechanism is pattern matching on tokens.

"Temperature controls creativity"

Temperature controls randomness in token selection. Low temp = pick the highest-probability token. High temp = more random selection. It's not "creativity" — it's how much the model deviates from its top prediction.

"LLMs are deterministic — same input, same output"

Only at temperature 0. At any temperature > 0, the sampling process introduces randomness, so outputs vary between calls.

Thinker, Not Calculator

Think of Claude as a **"thinker, not a calculator."**

A calculator gives the same answer every time. A thinker considers context, might phrase things differently, and can make mistakes — but handles novel situations no one explicitly programmed.

Understanding this shapes everything:

- **Prompts** guide the thinker's attention
- **Tools** give it real data it can't predict
- **Guardrails** catch its mistakes
- **Evaluation** measures quality probabilistically, not as pass/fail

Tokens — The Atoms of AI Communication

Every cost, every limit, every performance decision traces back to tokens. Learn what they are and why they matter.

Everything Traces Back to Tokens

Every interaction with Claude — every cost, every limit, every performance decision — traces back to **tokens**. Tokens are subword chunks that break text into reusable pieces, like syllables help children learn to read.

Common words like “the” are one token. Longer words like “understanding” become two tokens: `understand` + `ing`.

Tokens determine three critical things:

- **Cost** — how much you pay per API call
- **Limits** — whether your call succeeds (context window)
- **Speed** — how fast responses arrive

Syllables for Machines

Tokens solve the same problem syllables solve for reading. Imagine teaching a child to read by showing entire paragraphs — no letters, no syllables. Impossible.

Computers face the same problem. A string like “understanding” is just raw bytes with no structure. Tokens break text into reusable chunks of the right size: `understand` + `ing`.

The tokenizer learns these chunks statistically (byte-pair encoding) from training data, creating a vocabulary of ~100K pieces that can represent any text.

“Understanding tokenization”

↓

Understand ing token ization

= 4 tokens

Four Steps: Split, Map, Count, Fit

1. **SPLIT** — The tokenizer breaks your text into subword pieces using byte-pair encoding (BPE). Common words stay whole, rare words get split.
2. **MAP** — Each token maps to a numeric ID. “Claude” → 51423, “is” → 318, “helpful” → 10950. The model only sees numbers.
3. **COUNT** — Input tokens + output tokens = total tokens. You pay per token. Claude Sonnet: ~$3/M input, ~$15/M output.
4. **FIT** — All tokens must fit in the context window (200K tokens for Claude). Input + output share this space. Exceed it = hard error.

Rule of thumb: 1 token ~ 4 characters in English. 750 words ~ 1,000 tokens.

Counting Tokens and Checking Budget

```
// Counting tokens and checking budget
text = user_message + system_prompt
      + history
input_tokens = TOKENIZE(text).length

context_limit = 200000
max_output = 4096
available = context_limit - input_tokens

IF available < max_output:
  TRIM oldest messages from history
  WARN "Context window nearly full"

cost = (input_tokens * price_per_input)
     + (output_tokens * price_per_output)
LOG cost for monitoring
```

This pattern — tokenize, check budget, trim if needed, log cost — belongs in every production agent. Without it, you discover context overflow and cost overruns after they happen.

What People Get Wrong About Tokens

"Tokens are just words"

Not quite. Common words like “the” = 1 token. But “understanding” = 2 tokens. Punctuation and spaces can merge with surrounding text. The mapping is statistical, not dictionary-based.

"One token = one character"

In English, 1 token averages ~4 characters. But it varies wildly: a space = 1 token (1 char), “Claude” = 1 token (6 chars). Emojis can cost 2–3 tokens each.

"Tokens only matter for billing"

Billing is visible, but tokens also determine: whether your call succeeds (context limits), response speed (more tokens = more latency), and reasoning quality (critical info buried in a sea of tokens gets less attention).

Tokens Control Everything

The Numbers That Matter

A customer-support agent processing **50,000 calls/day** at 2,500 tokens each costs **~$700/day** on Sonnet.

A 20% token reduction saves **$4,200/month**.

Three habits for every agent builder:

- **Count tokens** — know your per-call budget before you hit production
- **Trim verbose prompts** — remove fluff from system prompts and history
- **Monitor context window usage** — log how close each call gets to the limit

MODULE

Prompts — The Language of Agent Control

Track 1: Foundations

MODULE 3 of 30

Prompts — The Language of Agent Control

10 min read

The Big Idea

Every message to Claude uses three roles: **system** (sets persona and rules), **user** (the question), and **assistant** (Claude's response).

The system prompt is the most powerful lever you have — it defines your agent's personality, capabilities, and constraints.

Prompt engineering patterns (**zero-shot**, **few-shot**, **chain-of-thought**) give you repeatable techniques to improve response quality without changing the model.

The Analogy

> Prompt Patterns as Teaching Strategies
> **Zero-shot** is a pop quiz — just ask the question and see if the student knows it.
>
> **Few-shot** is demonstrating solved problems before the test — show 2–5 examples, then ask.
>
> **Chain-of-thought** is a tutor working through a problem on a whiteboard step by step — the student follows the logic and makes fewer errors. Each strategy works best for different situations.

Zero-shot

Question
↓
Answer

Few-shot

Examples
↓
Question
↓
Answer

Chain-of-thought

Question
↓
Step 1
↓
Step 2
↓
Step 3
↓
Answer

How It Works

1
SYSTEM role — Sets the agent's persona, rules, and constraints. Loaded once, persists across the conversation. *"You are a security auditor. Only respond with JSON."*
2
USER role — The human's message. Each new question goes here.
3
ASSISTANT role — Claude's response. Included in conversation history for multi-turn context.
4
Messages array — Every API call sends the FULL conversation history. Claude has no built-in memory between calls — your code manages the array.
5
Stateless API — Each call is independent. Claude sees the messages array as if for the first time.

The Pseudocode

Three prompting patterns for the same question:

```
// ZERO-SHOT — just ask
prompt = "Classify this email as
  spam or not-spam:
  'You won $1M! Click here!'"

// FEW-SHOT — show examples first
prompt = "Examples:
  'Meeting at 3pm' → not-spam
  'Invoice attached' → not-spam
  'FREE VIAGRA!!!' → spam
  Now classify:
  'You won $1M! Click here!' →"

// CHAIN-OF-THOUGHT — reason step by step
prompt = "Classify as spam/not-spam.
  Think step by step:
  1. What signals suggest spam?
  2. What signals suggest legitimate?
  3. Weigh evidence. Final answer:"
```

Common Misconceptions

"The system prompt is just a greeting"

The system prompt is the most powerful control surface. A poorly structured one can drop accuracy from 92% to under 60%. Include: role, constraints, output format, and examples.

"Claude remembers previous conversations"

Claude is stateless. Every API call sends the full messages array. Between calls, nothing persists unless YOUR code saves it. There is no built-in memory.

"More words = better prompts"

Often the opposite. Verbose prompts waste tokens and can bury key instructions. Be specific and concise. Use XML delimiters to separate instructions from data.

Key Takeaway

Your agent's operating manual

The system prompt is your agent's operating manual — define role, rules, and output format clearly.

Use **zero-shot** for simple tasks, **few-shot** to teach by example, and **chain-of-thought** for multi-step reasoning (20–40% accuracy improvement on complex tasks).

Remember: Claude is stateless. Your code manages the conversation history.

Quick Quiz

Tap each card to reveal the answer.

Q1: Which prompting pattern improves accuracy most on multi-step math problems?

Tap to reveal

Answer: Chain-of-thought — by making Claude show its reasoning step by step, accuracy improves 20–40% on multi-step tasks.

Q2: Does Claude remember your previous conversation automatically?

Tap to reveal

Answer: No. Claude is stateless. Each API call sends the full messages array. Your code must manage conversation history.

Q3: What are the three roles in the Messages API?

Tap to reveal

Answer: System (agent's persona/rules), User (human's message), and Assistant (Claude's response).

MODULE

Context Engineering — Curating What the Model Sees

Track 1: Foundations

MODULE M03B · 5 of 32

Context Engineering — Curating What the Model Sees

10 min read

The Big Idea

Prompt engineering is writing the message you send. **Context engineering** is curating *everything* the model sees on each turn.

Every API call assembles a stack: system prompt + tool definitions + conversation history + retrieved documents + tool results + the user's current message.

The user message is often **under 5%** of what gets sent. Every byte competes for the same finite window — and the model has no privileged access to "the question."

The Analogy

> The Court Evidence Binder
> The prompt is the closing argument. The context is the entire evidence binder the jury reads — exhibits, depositions, prior testimony, expert reports.
>
> A new lawyer obsesses over the closing speech. An experienced lawyer spends most of their time curating the binder.
>
> Same trial, different verdict — because the binder shapes what the jury attends to before the speech ever begins.

Prompt Engineering

The argument
↓
1 message

Context Engineering

System
+
Tools
+
History
+
Retrieved
+
User turn

How It Works — The Four Levers

Every context decision is one of these four moves.

1

ADD — Include the content directly in the context. Cheap to access, expensive in tokens. Use for high-frequency, low-volume info.

2

COMPRESS — Replace verbose history with a summary. Use when older turns matter for continuity but not detail. Operationalized in M08.

3

RETRIEVE — Fetch only what's relevant per turn (RAG). Use for large reference sets where most facts won't apply. Operationalized in M09–M10.

4

OFFLOAD — Push subtasks to a subagent or external store. Returns only the final report. Operationalized in M14.

5

Static-first ordering — System prompt and tool defs don't change; history and retrieved chunks do. Putting static blocks *first* enables prompt caching (M22). Reverse the order and the cache misses every turn.

The Pseudocode

The `ContextBudget` pattern — account, decide, assemble.

```
// 1. ACCOUNT — layer-by-layer
LAYERS = {
  system_prompt,    // static
  tool_definitions, // static
  history,          // dynamic
  retrieved_chunks, // dynamic
  tool_results,     // dynamic
  user_turn         // dynamic
}
total = SUM(tokens(L) for L in LAYERS)

// 2. DECIDE — over budget?
IF total > budget:
  PICK a lever:
    - compress old history → summary
    - retrieve top-k only
    - offload subtask → subagent

// 3. ASSEMBLE — static FIRST
messages = [
  ...static_blocks,   // cache hits
  ...dynamic_blocks   // cache miss OK
]
SEND to Claude
```

Common Misconceptions

"Context engineering is just prompt engineering"

No. The prompt is one layer in the context stack. Context engineering covers the other five layers (system, tools, history, retrieval, tool results) plus the order they're stacked in.

"If I'm under the token limit, I'm fine"

No. Context rot — stale tool results, abandoned plans, resolved errors — degrades quality even at 60% of capacity. Signal-to-noise matters more than headroom.

"Order doesn't matter, content does"

Order is content's twin. Static-first ordering enables caching. The lost-in-the-middle effect means edge positions get more attention than middle ones. Where you put a fact changes whether the model uses it.

Key Takeaway

Curate the binder, not just the speech

Context engineering is the discipline of choosing what the model sees on each turn — under a finite token budget.

Four levers: **add**, **compress**, **retrieve**, **offload**. Static content first (for caching), dynamic content last.

Watch for **context rot**: stale info crowding out signal. A smaller, cleaner context often beats a larger, noisier one — even if you're under the limit.

Quick Quiz

Tap each card to reveal the answer.

Q1: Which is NOT one of the four context-engineering levers?

add · compress · retrieve · encrypt · offload

Tap to reveal

Answer: Encrypt. The four levers are add, compress, retrieve, and offload. Encryption is a security concern, not a context-curation move.

Q2: An agent works fine for 5 turns then degrades. Token usage is at 60% — well under the limit. Most likely cause?

Tap to reveal

Answer: Context rot. Stale tool results, abandoned plans, and superseded instructions accumulate, lowering signal-to-noise. The window isn't full but the noise floor is high. Compaction fixes it.

Q3: Where should you place the most important instruction in a long context?

Tap to reveal

Answer: At the edges (start or end), not the middle. Models attend more strongly to the beginning and end of the window — the lost-in-the-middle effect. Bullet 14 of 23 is forgotten.

Structured Output & Parsing

Agents produce data your code must parse, validate, and act on. Learn how to get reliable, structured output from Claude every time.

The Big Idea

Agents don't just generate text for humans — they produce **data that code must parse, validate, and act on**. Without structured output, your agent's responses are unpredictable text blobs that break downstream systems.

Claude's **tool use** feature solves this: by defining a JSON Schema, you force Claude to return structured data with labeled fields and typed values — like getting a filled-in spreadsheet instead of a rambling paragraph.

The difference in production is dramatic. Prompt-based JSON extraction fails **5–15%** of the time. Tool-use structured output fails **under 0.5%**. For a system handling 10,000 requests per day, that gap means the difference between hundreds of daily broken requests and near-zero failures.

The Analogy

> Everyday Analogy
> Imagine asking a coworker for customer data and they reply with a rambling paragraph: "Oh yeah, John works at Acme, his email is maybe john@acme.com, he might be a PM?" You'd have to parse that manually every time.
>
> Now imagine getting a **filled-in spreadsheet** instead — every field labeled, every value typed. That's the difference between unstructured and structured output. Tool use gives Claude a form to fill in, guaranteeing the structure.

Free text: "John works at Acme..."
↓ JSON.parse() ↓
Error!

vs.

Tool Use → schema
↓
{"name":"John", "company":"Acme", "email":"john@acme.com"}
Success!

How It Works

1
DEFINE a tool with a JSON Schema describing the output shape you want (field names, types, required fields).
2
SEND the message to Claude with the tool definition. Set `tool_choice` to force Claude to use it.
3
RECEIVE a `tool_use` response with structured data guaranteed to match your schema.
4
VALIDATE the data with Pydantic (Python) or Zod (TypeScript) for extra safety.
5
RETRY on failure — if validation fails, send the error back to Claude and ask it to fix the response.

Tool use for structured output is a "trick" — you define a tool not because you want to call a function, but because the JSON Schema forces Claude to produce structured data.

The Pseudocode

A complete extraction pipeline with validation and retry:

```
// Define a "tool" that's really
// an output schema
tool = DEFINE "extract_contact":
  input:
    name    (string, required)
    email   (string, required)
    company (string)
    role    (string)

// Force Claude to use the tool
response = SEND message to Claude
  tools: [extract_contact]
  tool_choice: "extract_contact"

// Parse the structured response
data = response.tool_use.input
VALIDATE data against schema

IF validation fails:
  SEND error back to Claude
  ASK Claude to fix it
  (retry up to 3 times)
```

Common Misconceptions

"Just ask Claude to return JSON in the prompt."

Fragile. Claude might add markdown formatting, forget fields, use wrong types, or slip in conversational text. Tool use with a schema guarantees structure — Claude physically cannot return free-form text.

"Structured output means I don't need validation."

Tool use guarantees valid JSON structure, but not semantic correctness. An email field will be a string, but it might not be a valid email. Always validate with Pydantic or Zod.

"If the response is wrong, just retry the same call."

Better: send the validation error back to Claude so it can fix the specific issue. Error-aware re-prompting succeeds more often than blind retries.

Key Takeaway

Remember This

Structured output is the **contract between AI and your code**. Use tool use with JSON Schema to force Claude into predictable data shapes. Always validate with Pydantic or Zod. Always implement error-aware retry (send the error back to Claude).

Teams switching from prompt-based JSON to tool use typically see parse failures drop from **5–15%** to **under 0.5%**.

Quick Quiz

Tap each question to reveal the answer.

Q1: Why use tool use for structured output instead of asking Claude to "return JSON"?

Tap to reveal

Tool use with a JSON Schema guarantees the output structure. Prompt-based JSON can include markdown, missing fields, or conversational text mixed in.

Q2: Does tool use guarantee the data is semantically correct?

Tap to reveal

No. It guarantees valid JSON structure matching your schema, but not semantic correctness. An "email" field will be a string, but might not be a valid email. Validate separately.

Q3: What should you do when validation fails?

Tap to reveal

Send the validation error back to Claude and ask it to fix the specific issue (error-aware re-prompting), rather than blindly retrying the same call.

Function Calling — Giving Claude Hands

The core mechanism that turns a chatbot into an agent: Claude decides what to do, your code does it.

The Big Idea

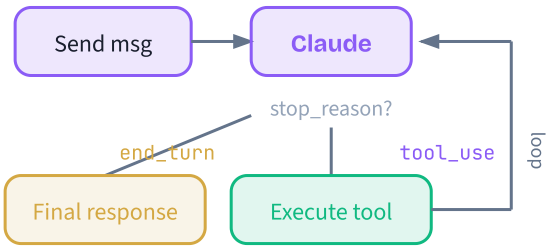
Function calling (tool use) is the core mechanism that turns a chatbot into an agent. You define tools as JSON objects with a name, description, and input schema. Claude reads these definitions, decides when a tool is needed, and returns a structured request: "call this function with these arguments."

Your code executes the function and sends results back. This loop — Claude decides, you execute, Claude continues — is the heart of every agent.

Without function calling, Claude can only talk. With it, Claude can check the weather, query databases, send emails, and take real-world action — all while reasoning about when and why each action is appropriate.

The Analogy

> Everyday Analogy
> **Think of Claude as a doctor and tools as lab tests.** The doctor (Claude) examines the patient (reads the user's message), decides which test to order (selects a tool), and writes a lab order form (returns a `tool_use` response with structured arguments).
>
> **The lab (your code) runs the test and sends results back.** The doctor reads the results and either orders another test or gives the diagnosis (final answer).
>
> **Claude never runs the tests directly — it only decides which ones to order.** This separation is fundamental: Claude reasons and decides, your code acts and returns data.



How It Works

1
Define tools as JSON with name, description, and `input_schema` (JSON Schema). The description is the most important part — Claude picks tools based on descriptions, not names.
2
Send the user's message plus tool definitions to Claude.
3
Check `stop_reason`: if `"tool_use"`, Claude wants to call a tool. If `"end_turn"`, Claude is done.
4
Execute the requested tool with the provided arguments (your code runs the actual function).
5
Append the tool result to the messages array and send back to Claude.
6
Repeat until Claude returns `"end_turn"` with a final text response.

The Pseudocode

The universal agent loop pattern:

```
// Define tools for Claude
tools = [
  DEFINE "get_weather":
    description: "Get current weather
      for a city. Returns temp,
      conditions, humidity."
    input: city (string, required)
  ,
  DEFINE "search_database":
    description: "Search product catalog
      by query. Returns matching
      products with prices."
    input: query (string, required),
           limit (int, default 5)
]

// The agent loop
messages = [user_question]
WHILE true:
  response = SEND messages, tools
             to Claude
  IF response.stop_reason == "tool_use":
    tool = FIND matching function
    result = EXECUTE tool(response.input)
    APPEND tool_result to messages
  ELSE:
    RETURN response.text
    BREAK
```

Key insight: the `WHILE` loop checking `stop_reason` is the universal agent pattern. A single user question can trigger multiple tool calls in sequence — the loop handles this automatically.

Common Misconceptions

"Claude executes the tools directly."

No. Claude returns a structured request saying "please call this function with these arguments." Your code runs the actual function. Claude never has direct access to your systems.

"Tool names are what Claude uses to pick tools."

Claude relies heavily on the *description*, not the name. A one-word description like "Search" gives Claude nothing to work with. Good descriptions include: what it does, what data source, output format, and when to use it.

"One user question = one tool call."

A single question can trigger multiple tool calls in sequence. "What's the weather in Tokyo and New York?" might trigger `get_weather` twice. The loop handles this automatically.

Key Takeaway

> **Remember This**
>
> Function calling is the bridge between AI reasoning and real-world action. Claude decides **what** to do; your code does it.
>
> The tool description is your **#1 lever** for accuracy — spend more time writing descriptions than implementations.
>
> The `while` loop checking `stop_reason` is the universal agent pattern you'll use throughout this course. Every agent you build in M06 through M15B uses this same loop at its core.

Quick Quiz

Tap each question to reveal the answer.

Q1: Who actually executes the tool functions — Claude or your code?
Tap to reveal
Your code. Claude only returns a structured request specifying which tool to call and with what arguments. Your code runs the actual function.
Q2: What's the most important part of a tool definition for selection accuracy?
Tap to reveal
The description. Claude picks tools based on descriptions, not names. Include: what it does, data source, output format, and when to use it vs. alternatives.
Q3: What is stop_reason "tool_use" vs "end_turn"?
Tap to reveal
`"tool_use"` means Claude wants to call a tool — execute it and loop back. `"end_turn"` means Claude is done and has a final text response for the user.

Multi-Tool Orchestration

Run multiple tools in parallel, chain their outputs sequentially, and handle failures gracefully — turning a single-tool agent into a powerful multi-tool system.

Track 2: Tool Use
Module 6 of 30

Multi-Tool Orchestration

Run multiple tools in parallel, chain their outputs sequentially, and handle failures gracefully — turning a single-tool agent into a powerful multi-tool system.

From One Tool to Many

In M05 you gave Claude **one tool at a time**. Real agents need multiple tools working together. Multi-tool orchestration is the art of deciding which tools to run, in what order, and what to do when things break.

There are three core patterns:

- **Parallel execution** — run independent tools at the same time to cut latency
- **Sequential chaining** — pipe one tool's output into the next tool's input
- **Dynamic registration** — only send relevant tools per request to improve accuracy and reduce token cost

Claude handles the hard part: when it returns multiple **tool_use** blocks in one response, those tools are independent and safe to run in parallel. When it returns one at a time, that means it needs the result before deciding the next step.

The Kitchen Brigade

> Imagine a kitchen where only one sous chef handles all prep work — chopping onions, then dicing tomatoes, then mincing garlic, one task after another while the head chef waits idle.

> Dinner service grinds to a halt. Three 10-minute tasks take 30 minutes total, and every dish that depends on those ingredients sits idle the entire time.

> Now picture three sous chefs working simultaneously — one chops onions, one dices tomatoes, one minces garlic. All three finish in 10 minutes instead of 30. That is **parallel tool execution**.
>
> But some tasks must stay sequential: you can't plate a dish before it's cooked. That is a **sequential chain** — search produces URLs, fetch retrieves pages, summarize distills text. Each stage needs the previous stage's output.

Orchestration in 5 Steps

1. **Define tools** — Register each tool with a clear name, description, and input schema. Good descriptions are critical: Claude picks tools based on what the description says.
2. **Send request** — Pass the user's message and the tools array to Claude. Claude analyzes the task and decides which tools to call.
3. **Check the response** — If Claude returns tool_use blocks, execute them. Multiple blocks in one response means they are independent: run them in parallel.
4. **Return results** — Send all tool_result blocks back in a single user message, each matched to its tool_use_id. If a tool failed, set is_error: true with a descriptive message so Claude can adapt.
5. **Loop until done** — Repeat steps 2-4 until Claude responds with end_turn (no more tool calls). This agentic loop is what turns a chatbot into an agent.

Multi-Tool Agent Loop

This pseudocode shows the core orchestration pattern: loop, detect tool calls, execute in parallel, feed results back.

```
DEFINE tools = [search, fetch, summarize]

LOOP:
  response = call_claude(messages, tools)

  IF response.stop_reason == "end_turn"
    RETURN response.text

  tool_calls = response.tool_use_blocks

  // Run all calls in parallel
  results = PARALLEL FOR EACH call IN tool_calls:
    TRY:
      output = execute(call.name, call.input)
      YIELD {id: call.id, content: output}
    CATCH error:
      YIELD {id: call.id, is_error: true,
             content: error.message}

  APPEND assistant response to messages
  APPEND all results to messages
  CONTINUE LOOP
```

Key insight: Claude decides the execution pattern. Multiple tool_use blocks = parallel. One block at a time = sequential chain.

What People Get Wrong

"More tools = more capable agent"
Actually the opposite. Tool selection accuracy degrades past 5-6 tools. Each extra tool adds ambiguity and burns input tokens. An agent with 4 focused tools outperforms one with 18 scattered tools. Use dynamic registration to filter the tools array per request.

"Sequential is always worse than parallel"
Not at all. When Tool B needs Tool A's output (e.g., fetch a page from a URL that search returned), they MUST run sequentially. Forcing parallel execution on dependent tools means Tool B runs with no input and produces garbage. Parallelize independent tools, chain dependent ones.

"Retrying a failed tool 100 times will eventually work"
Brute-force retries burn tokens and time. If an endpoint is down, it stays down. Use exponential backoff (wait 1s, 2s, 4s) with a max of 3 retries for transient failures. For persistent failures, return is_error: true and let Claude pick an alternative tool or inform the user.

Remember This

The Orchestration Triangle

Every multi-tool agent balances three forces:

- **Parallelism** — run independent tools simultaneously to cut wall-clock time. Three 10-second tools finish in 10 seconds, not 30.
- **Sequencing** — chain dependent tools so each stage gets valid input. A 3-step chain (search, fetch, summarize) requires 4 API round trips total.
- **Resilience** — report errors with is_error: true so Claude can adapt. Never silently swallow failures or retry endlessly.

And one bonus principle: **keep the tool count low**. Dynamic registration (filtering tools per request) saves ~6,000 tokens when going from 20 tools to 5, and improves selection accuracy at the same time.

Test Your Understanding

Q1: Tool A and C are independent, but B needs A's result. What is the optimal execution order?

Answer: Run A and C in parallel, then run B after A completes. This gives maximum parallelism while respecting the data dependency between A and B.

Q2: A tool fails with a network timeout. What is the best way to report this to Claude?

Answer: Return a tool_result with is_error: true and a descriptive message explaining the failure. This lets Claude reason about alternatives — it might try a different tool or inform the user. Never crash the loop or silently retry forever.

Q3: You have 20 tools at 400 tokens each. Filtering to 5 per request saves how many tokens?

Answer: 6,000 tokens per request. 20 x 400 = 8,000 tokens. 5 x 400 = 2,000 tokens. That is 6,000 tokens saved on every single API call — real cost savings at scale, plus improved tool selection accuracy.

MCP — Model Context Protocol

The universal standard that lets AI models connect to any tool or data source through one protocol — no custom integrations needed.

Track 2: Tool Use
Module 7 of 30

MCP — Model Context Protocol

The universal standard that lets AI models connect to any tool or data source through one protocol — no custom integrations needed.

One Protocol to Connect Them All

Before MCP, connecting an AI model to N tools required N custom integrations. If you had 4 AI apps and 5 tools, that meant **20 separate adapters** (4 × 5). Each adapter had its own authentication, serialization, and error handling.

MCP collapses this to **4 + 5 = 9** standardized connections. Each AI app implements one MCP client; each tool implements one MCP server. They all speak the same protocol — JSON-RPC 2.0 messages over stdio (local) or HTTP+SSE (remote).

The result: any MCP client can talk to any MCP server, just like any USB-C device works with any USB-C port. Tool authors write their server once, and every compatible AI application can use it immediately.

The USB-C Moment for AI

> Remember when every phone had its own charger? Micro-USB, Lightning, barrel jacks — a tangled drawer of cables. Every new device meant buying yet another proprietary cable.

> AI integrations had the same problem. Every tool needed bespoke glue code for every AI model. Switching models or adding tools meant rewriting adapters from scratch. The integration tax grew with every new combination.

> MCP is USB-C for AI. One universal plug. The AI app is the device, the tool is the accessory, and MCP is the standard cable. Plug in any server, and the client discovers its capabilities automatically — no driver installs, no custom wiring.

Architecture & Handshake

MCP has two sides: a **client** (the AI app, like Claude Desktop) and a **server** (your tool or data source). They connect through a three-step handshake:

1. **Initialize:** The client sends an `initialize` request with its protocol version and capabilities.
2. **Capabilities response:** The server replies with what it offers — tools (actions), resources (read-only data), and prompts (workflow templates).
3. **Confirm:** The client sends `initialized` to acknowledge, and the connection is live.

After the handshake, the client can call tools, read resources, or use prompt templates. All messages are JSON-RPC 2.0 — a simple `{"method": "tools/call", "params": {...}}` format.

Three primitives organize what servers expose:

- **Resources** — read-only data (like a file listing or DB schema). No side effects.
- **Tools** — actions that do something (write a file, send an email). Require model confirmation.
- **Prompts** — reusable workflow templates that guide the model through multi-step tasks.

Building an MCP Server

Here is the core pattern for creating an MCP server that exposes a tool. This works the same regardless of language:

```
server = CREATE_MCP_SERVER("my-file-tools")

REGISTER_TOOL(server,
  name: "read_file",
  description: "Read contents of a file",
  inputs: { path: STRING (required) },
  handler: FUNCTION(params)
    content = FILE_READ(params.path)
    IF error THEN RETURN error_response
    RETURN text_response(content)
  END
)

REGISTER_RESOURCE(server,
  uri: "files://listing",
  handler: RETURN directory_listing("/data")
)

server.START(transport: "stdio")
```

The client (Claude Desktop) discovers this server via a config file that specifies the command to launch it. Once connected, Claude can call `read_file` or browse the directory listing — all through the standard MCP protocol.

What MCP Is NOT

"MCP is just another API framework like REST"
REST and GraphQL are general-purpose web API standards. MCP is specifically a protocol for connecting AI models to tools. It includes capability discovery, primitive categorization (Resources, Tools, Prompts), and a negotiation handshake — none of which REST provides out of the box.

"MCP replaces function calling"
MCP *uses* function calling under the hood. When Claude calls an MCP tool, the Messages API still uses `tool_use` blocks internally. MCP adds a standardized discovery and transport layer on top, so tools from different servers compose without custom glue code.

"MCP servers must be hosted on the internet"
Most MCP servers run as local subprocesses on your own machine. The stdio transport means the client just spawns your script and communicates through stdin/stdout pipes — no network, no authentication needed. Remote hosting via HTTP+SSE is available but is the exception, not the norm.

Remember This

MCP = Universal Plug for AI Tools

MCP turns the N × M integration problem into N + M. Instead of building a custom adapter for every model-tool combination, you build one MCP server per tool and one MCP client per AI app. They discover each other's capabilities automatically through a standard handshake.

The three primitives keep things organized: **Resources** for read-only data, **Tools** for actions with side effects, **Prompts** for reusable workflow templates. Choosing the right primitive saves tokens and makes your server easier to use.

Start local with stdio transport. Add a tool, register it, point Claude Desktop at it with a config file, and you have a working MCP integration in minutes.

Test Your Understanding

Q1: Which MCP primitive should you use to expose a database table's schema for browsing?

Answer: Resource. A database schema is read-only data with no side effects. Resources are designed for exactly this — data the client can browse without triggering any actions. Using a Tool would waste tokens on unnecessary tool-use framing.

Q2: What transport should you use for an MCP server running on a remote cloud VM?

Answer: HTTP+SSE. The stdio transport requires the client to launch the server as a local subprocess, which is not possible over a network. HTTP+SSE is the MCP transport designed for remote server connections.

Q3: What is the correct order of the MCP initialization handshake?

Answer: Client sends initialize → Server responds with capabilities → Client sends initialized. The client always initiates, the server declares what it offers, and the client confirms to complete the connection.

Conversation Management

How to give a stateless API the illusion of memory — and keep costs under control as conversations grow.

Track 3: Memory & Context
Module 8 of 30

Conversation Management

How to give a stateless API the illusion of memory — and keep costs under control as conversations grow.

Claude Has No Memory

Every API call to Claude is **completely stateless**. There is no session, no server-side storage, no memory of any kind between requests. The only way to give Claude "memory" is to send the entire conversation history with every single request.

This means **your code is the memory**. You decide what Claude remembers by choosing which messages to include in the `messages` array. Include everything and costs balloon. Include too little and Claude loses critical context.

A **conversation manager** solves this by sitting between your app and the API. It stores every exchange, tracks token usage, automatically compresses old messages, and ensures Claude always gets the right briefing — recent details in full, older context summarized, and critical facts pinned.

The Expert with Perfect Amnesia

> Imagine you hire a world-class expert for advice, but they have perfect amnesia — every time you walk into the room, they have zero memory of any previous conversation.

> If you just said "so what do you think about option B?" they would stare blankly, because they have no idea what option A was, who you are, or what problem you are solving. Every interaction starts from absolute zero.

> This is exactly how the Claude Messages API works. Each API call is a fresh room with a fresh expert. The only way to give Claude "memory" is to hand it a written transcript of every previous exchange — your code replays the entire conversation from scratch on every single request.

Three History Strategies

As conversations grow, you need a strategy for what to include in each API call:

1. **Full History** — send every message every time. Simple, but hits token limits and inflates costs quickly. Best for short conversations under 20 turns.
2. **Sliding Window** — keep only the last N messages. Cheap and fast, but loses early context. If the user's account number was in message 1, it vanishes by turn N+1.
3. **Summarization** — compress older messages into a short summary using Claude itself, then prepend it to recent messages. Preserves meaning while staying within token budgets.

Most production agents use a **hybrid approach**: a summary of old turns + full recent turns + "pinned facts" (account numbers, key decisions) that are never summarized.

Token budget formula: `available_history = context_window - system_prompt - current_message - max_tokens`

Conversation Manager

A production conversation manager handles storage, token counting, pruning, summarization, and serialization:

```
CREATE manager WITH:
  system_prompt, token_threshold,
  recent_turns = 4

ON send(new_message):
  APPEND new_message TO history
  total = COUNT_TOKENS(history)

  IF total > token_threshold:
    old = history[0 .. end - recent_turns]
    summary = CALL_CLAUDE("Summarize:", old)
    REPLACE old WITH summary
    KEEP last recent_turns verbatim

  CALL Claude API WITH system_prompt
    + history + new_message
  APPEND response TO history

ON save(path): WRITE history TO JSON file
ON load(path): READ history FROM JSON file
```

Key detail: after pruning, verify messages still alternate user/assistant — the API rejects malformed arrays.

Common Mistakes

"Claude remembers our previous conversation."
No. Each API call is completely stateless. There is no session, no server-side storage. The "memory" you experience in chat interfaces like claude.ai is built by the client replaying message history — the exact technique you are learning here.

"Longer context = better results."
Not necessarily. Research shows recall degrades with context length — the "lost in the middle" effect. Information buried in the middle of a very long context gets lower attention than information at the start or end. Cramming everything into a 200K payload can make Claude worse at finding relevant facts.

"Summarization is lossless."
No. Summarization is lossy compression. It preserves the gist but loses specifics: exact names, account numbers, dates, dollar amounts. That is why production systems use "pinned facts" blocks that are never summarized — critical specifics stay verbatim while surrounding context gets compressed.

Remember This

Your Code IS the Memory

Claude has zero memory between API calls. Every "memory" the model has comes from the messages array your code sends. A conversation manager is the essential middleware that decides what Claude remembers and what gets compressed or dropped.

Cost Grows With Every Turn

A 50-turn support conversation with full history sends ~75,000 tokens per request. At $3/M input tokens, that is $0.23 per reply. Across 10,000 daily conversations, that is $2,300/day. Summarization with the last 4 turns cuts that to ~$0.03/reply ($300/day) — an 87% cost reduction while preserving context.

Smart Pruning Preserves What Matters

Score messages by importance. Greetings and filler get cut first. Key facts (account numbers, decisions, tool results) are preserved no matter how old they are. Always prune in user/assistant pairs and check for downstream references.

Test Yourself

1. What happens if you send an API request to Claude without including previous messages?

Answer: Claude treats it as a brand-new conversation with no memory of anything prior. The API is fully stateless — there is no session storage or caching between requests. If you omit previous messages, they simply do not exist for Claude.

2. A support bot must remember the user's account number from message 1 across 50+ turns. Which strategy is best?

Answer: Summarization with importance-based retention. Summarize old turns but always preserve key facts like account numbers in a "pinned facts" block. A pure sliding window would silently drop the account number after N turns. Full history works but wastes tokens and money at 50+ turns.

3. Why does summarization use "pinned facts" that are never compressed?

Answer: Because summarization is lossy compression — it preserves the gist but can paraphrase away exact numbers, IDs, dates, and names. Pinned facts ensure critical specifics (account numbers, diagnosis codes, order IDs) survive every summarization cycle verbatim, even as surrounding context gets compressed.

RAG — Retrieval-Augmented Generation

Give Claude access to your documents so it can answer with real, up-to-date facts instead of guessing.

Track 3: Memory & Context
MODULE 9 of 30

RAG — Retrieval-Augmented Generation

Give Claude access to your documents so it can answer with real, up-to-date facts instead of guessing.

Search First, Then Generate

Claude's training data has a cutoff — it doesn't know **your** documents, your company wiki, or last week's policy update. RAG solves this by **searching your documents** and pasting the relevant parts into Claude's prompt before it answers.

Think of it as giving Claude a **cheat sheet customized for each question**. The model itself doesn't change — you just change what it can see when answering.

This is the most practical way to make Claude useful for domain-specific work without expensive fine-tuning or retraining. If your data changes weekly, RAG picks up the changes automatically because it searches at query time.

The Open-Book Exam

> Imagine a closed-book exam. The student (Claude) can only use what they memorized during training. If the answer is in a document they never studied, they're stuck — or worse, they guess confidently and get it wrong.

> Your company's internal policies, product specs, and customer data were never in Claude's training set. Asking Claude about them is like asking a student about a chapter they skipped.

> Now it's an **open-book exam**. Before answering each question, the student flips to the most relevant pages and reads them. RAG is the page-flipping step — it finds the right pages so Claude can read them before responding.

Three Steps to RAG

1. **PREPARE** (run once) — Split your documents into small chunks (roughly 200–500 words each). Convert each chunk into an embedding — a number-array that captures its meaning. Store all embeddings in a vector database.
2. **SEARCH** (every question) — Convert the user's question into an embedding using the same model. Search the vector database for the 3–5 chunks whose embeddings are closest to the question's embedding.
3. **GENERATE** — Paste those top chunks into Claude's prompt alongside the user's question. Claude reads the provided context and answers based on the actual document content, not guesswork.

That's it: **split, search, stuff into prompt**. The "augmented" in RAG means the generation step is augmented with retrieved context.

RAG in Pseudocode

```
// SETUP (run once)
FOR each document:
  chunks = SPLIT document into ~500-word pieces
  FOR each chunk:
    embedding = EMBED(chunk)
    STORE(embedding, chunk) in vector DB

// QUERY (run per question)
query_embedding = EMBED(user_question)
top_chunks = SEARCH vector DB
  for nearest to query_embedding (limit 3)

prompt = "Answer using ONLY this context:\n"
  + top_chunks + "\n" + user_question
response = ASK Claude with prompt
```

Key insight: The EMBED function turns text into numbers so that similar meanings end up near each other in the number space. "Heart attack symptoms" and "signs of myocardial infarction" would produce nearby embeddings even though they share zero words.

Common Misconceptions

❌ **"RAG is the same as fine-tuning"**
Fine-tuning permanently changes the model's weights — it costs thousands of dollars and takes weeks. RAG doesn't touch the model at all. It just changes what text is in the prompt. You can update your documents instantly without retraining anything.

❌ **"Bigger chunks are better"**
Usually the opposite. A 2,000-word chunk dilutes the one relevant sentence with pages of noise. Smaller chunks (200–500 words) give more precise search results. If you need surrounding context, use overlapping chunks instead of giant ones.

❌ **"RAG eliminates hallucinations"**
RAG *reduces* hallucinations significantly but doesn't eliminate them. Claude can still misinterpret the retrieved context, combine fragments incorrectly, or fill gaps with plausible-sounding fiction. Always validate critical outputs.

Key Takeaway

RAG = Search + Generate

You search your documents for relevant pieces, paste them into the prompt, and let Claude answer from that context. No model changes needed, no retraining, no fine-tuning.

The three steps are always the same: **chunk your docs, embed and store them, then search at query time**. Everything else — chunk size tuning, re-ranking, hybrid search — is optimization on top of this core pattern.

Next up in **Module 10**: Advanced RAG Patterns — hybrid search, re-ranking, query expansion, and multi-step retrieval to make your RAG pipeline production-ready.

Quick Quiz

Q1: Does RAG change Claude's model weights?

Answer: No. RAG only changes what text is included in the prompt. The model's weights remain untouched. This is what makes RAG so much cheaper and faster than fine-tuning.

Q2: Why split documents into small chunks instead of sending the whole document?

Answer: Two reasons: (1) Embedding models have token limits and work best on shorter text. (2) Smaller chunks give more precise search results — you retrieve only the relevant paragraph, not 50 pages of noise.

Q3: What is the main risk that RAG does NOT fully solve?

Answer: Hallucination. Claude can still misinterpret or incorrectly combine the retrieved context. RAG significantly reduces hallucinations by grounding answers in real documents, but it does not eliminate them entirely.

Advanced RAG Patterns

Go beyond basic retrieve-and-generate with hybrid search, re-ranking, query transformation, and evaluation metrics.

Track 3: Memory & Context
Module 10 of 30

Advanced RAG Patterns

Go beyond basic retrieve-and-generate with hybrid search, re-ranking, query transformation, and evaluation metrics.

~10 min read

Why Basic RAG Hits a Ceiling

Naive RAG -- chunk, embed, retrieve by cosine similarity, generate -- works for simple use cases. But production systems hit a ceiling fast: vector search misses exact terms, rough similarity scores rank documents poorly, vague queries retrieve irrelevant context, and noisy chunks dilute answer quality.

Advanced RAG wraps the basic pipeline with three optimization layers, each targeting a different failure mode. **Pre-retrieval** transforms the query before searching (asking a better question). **Retrieval enhancement** combines keyword and semantic search, then re-ranks results by true relevance. **Post-retrieval processing** compresses chunks to remove noise and verifies citations. Each layer can be added independently -- the right combination depends on your data and queries.

Benchmarks show naive RAG achieves 55-65% answer accuracy. Adding hybrid search lifts that to 70-75%. Adding re-ranking pushes it to 80-85%. Query transformation on top gets you to 85-90%. For a bot handling 10,000 queries/day, going from 60% to 85% accuracy means **2,500 fewer wrong answers every day**.

The Research Librarian

> Imagine searching Google by typing your exact question and reading only the first result -- sometimes it is exactly right, but often it is tangential, outdated, or misses the nuance of what you really needed.

> You waste time reading irrelevant pages, miss the one article that actually answers your question because it used different wording, and have no way to know if the answer you got is actually trustworthy.

> Advanced RAG is like having a research librarian who first **rephrases your question three different ways** (query transformation), searches both a **keyword index and a topic index** (hybrid search), carefully reads the top candidates to **rank them by actual relevance** (re-ranking), **highlights only the sentences that matter** (compression), and then gives you a sourced answer. Every stage the librarian adds makes the final answer more accurate.

The Advanced RAG Pipeline

1. **Query Transformation** -- Rephrase the user's query before searching. HyDE generates a hypothetical answer to embed (better vector matches). Multi-query tries 3-5 reformulations (broader recall). Step-back abstracts to a general question first (better context).
2. **Hybrid Search** -- Run BM25 keyword search and vector similarity search in parallel. BM25 catches exact IDs and proper nouns that vectors miss. Vectors catch semantic meaning that keywords miss.
3. **Reciprocal Rank Fusion** -- Merge the two ranked lists using RRF: score = 1/(k + rank) for each list. Uses rank positions, not raw scores, so you never need to normalize BM25 scores against cosine similarities.
4. **Re-Ranking** -- A cross-encoder (or Claude itself) reads each query-document pair together and scores true relevance 1-10. This is slower but dramatically more accurate than cosine similarity. Apply to the top 20-50 candidates only.
5. **Contextual Compression** -- Extract only the query-relevant sentences from each chunk before feeding to the generator. Reduces token cost (often 80%+ savings) and removes noise that causes hallucination.

Advanced RAG Pipeline

```
FUNCTION advanced_rag(user_query):
  // Step 1: Query transformation
  hypothesis = LLM("Write a paragraph
    that answers: " + user_query)
  queries = [user_query, hypothesis]

  // Step 2: Hybrid search
  bm25_results = BM25_search(user_query)
  vector_results = vector_search(hypothesis)

  // Step 3: Fuse with RRF
  fused = reciprocal_rank_fusion(
    [bm25_results, vector_results], k=60)

  // Step 4: Re-rank top candidates
  top_20 = fused[:20]
  FOR each doc IN top_20:
    doc.score = LLM("Rate relevance
      of doc to query, 1-10")
  ranked = SORT(top_20, by score DESC)

  // Step 5: Compress & generate
  context = ""
  FOR each doc IN ranked[:5]:
    context += LLM("Extract only
      query-relevant sentences", doc)
  RETURN LLM(user_query, context)
```

What People Get Wrong

"Hybrid search is always better than pure semantic search."
Not always. BM25 wins for exact IDs, codes, and proper nouns. But for abstract or conceptual queries ("explain debtor risk factors"), vector-only search can outperform hybrid because BM25 adds noise from irrelevant keyword hits. Always test with your actual queries.

"Re-ranking is free improvement."
It adds 100-300ms of latency and costs an extra LLM call per query. For simple factual lookups where initial retrieval is already good, re-ranking just slows things down. Reserve it for complex or high-stakes queries where ranking accuracy justifies the cost.

"More retrieval stages = better quality."
Each stage adds latency and complexity. In practice, you see diminishing returns after 2-3 stages. Hybrid search + re-ranking covers most use cases. Adding query transformation, compression, AND re-ranking may gain 2-3% accuracy while tripling your latency and cost.

Remember This

Each advanced RAG pattern targets a specific failure mode

Hybrid search fixes keyword blindness -- when vector search misses exact terms like invoice numbers or product codes.

Re-ranking fixes rough ordering -- when the truly relevant document is buried at position #8 instead of position #1.

Query transformation fixes poor input -- when vague or ambiguous questions lead to bad retrieval no matter how good your search is.

Contextual compression fixes noisy context -- when retrieved chunks contain 90% irrelevant text that distracts the generator.

You do not need all four. Diagnose your pipeline's specific weakness using precision, recall, and faithfulness metrics, then add only the patterns that address it. Start with hybrid search + re-ranking -- they cover the most common failure modes with the best cost-to-quality ratio.

Test Your Understanding

1. When would BM25 keyword search outperform vector similarity search?

Answer: When the query contains specific identifiers like invoice numbers, product codes, or proper nouns. BM25 finds exact string matches that vector search might miss because it focuses on semantic meaning rather than literal terms.

2. What problem does Reciprocal Rank Fusion (RRF) solve?

Answer: RRF merges ranked lists from different search systems (like BM25 and vector search) without needing to normalize their incompatible scores. It uses rank positions instead of raw scores, so a BM25 score of 12.4 and a cosine similarity of 0.87 can be compared fairly.

3. A RAG system has high recall but low precision. What does this mean, and which pattern helps most?

Answer: It means the system retrieves relevant documents but also too many irrelevant ones. **Re-ranking** helps most here -- it applies a more powerful model to sort the candidates by true relevance, pushing irrelevant results down the list so only the best documents reach the generator.

Multi-Layer Memory

Give your agent a brain that remembers — across turns, across sessions, across tasks.

Track 3: Memory & Context
Module 11 of 30

Multi-Layer Memory

Give your agent a brain that remembers — across turns, across sessions, across tasks.

One Memory Type Is Not Enough

Production agents need **three specialized memory tiers**, each optimized for a different kind of information:

- **Working memory** — a fast, mutable scratchpad for the current task. Holds user intent, extracted entities, intermediate tool results, and the plan. Lives in RAM, cleared when the task finishes.
- **Episodic memory** — a searchable archive of past interactions. Stores summarized conversation records in a vector database, retrieved via semantic similarity search.
- **Procedural memory** — a library of reusable action sequences. Stores proven tool-call chains as templates so the agent can reuse workflows instead of reasoning from scratch.

A support agent handling 500 conversations/day generates ~2 million tokens of history per week. Without separated tiers, you hit context limits or pay $60+/day in wasted API costs. Multi-layer memory keeps the prompt lean: typically under 4,000 tokens of memory context per call.

Your Brain Already Does This

> Imagine if your brain stored everything in one place — today's grocery list, how to ride a bike, your childhood memories, and the recipe for your favorite pasta — all jumbled in a single notebook.

> You'd waste enormous time searching, the notebook would fill up fast, and important long-term memories would get crowded out by temporary notes. You'd forget how to ride a bike because you overwrote it with a shopping list.

> Your brain uses different systems for a reason: **working memory** for what you're thinking right now (holding a phone number while dialing), **episodic memory** for past experiences (what happened at your wedding), and **procedural memory** for learned skills (how to ride a bike). AI agents need the same separation — a single context window is the "one notebook" approach, and it breaks down at scale.

Memory Architecture in 6 Steps

1. **Request arrives.** The Memory Manager creates a working memory scratchpad and loads relevant episodic and procedural memories.
2. **Working memory tracks state.** As the agent processes the request, it writes intent, entities, and tool results to the scratchpad — injected into every LLM call.
3. **Episodic memory retrieves context.** The agent searches the vector database for the 2-3 most relevant past conversation summaries and adds them to the prompt.
4. **Procedural memory suggests a plan.** If a stored action template matches the current task, the agent retrieves and follows it instead of reasoning from scratch.
5. **Summarize on exit.** When the conversation ends, a summarization pipeline compresses it from ~4,000 tokens to ~300 tokens and stores the summary as a new episode.
6. **Persist everything.** Working memory flushes to Redis or SQLite. Episodic memory writes embeddings to ChromaDB with persistent storage. Procedural memory saves templates to a relational table.

Memory Manager Overview

```
class MemoryManager:
  working  = {}          # current task state
  episodic = VectorDB()  # past conversation summaries
  procedural = DB()      # reusable action templates

  on_request(user_msg):
    working["intent"] = parse(user_msg)
    working["entities"] = extract(user_msg)
    episodes = episodic.search(user_msg, top_k=3)
    procedure = procedural.match(working["intent"])
    prompt = build(working, episodes, procedure)
    result = llm.call(prompt)
    working["result"] = result
    return result

  on_session_end(conversation):
    summary = llm.summarize(conversation)
    episodic.store(embed(summary), metadata)
    working.clear()
```

Each tier has a single job: working memory tracks the “now,” episodic memory recalls the “before,” and procedural memory provides the “how.” The Memory Manager orchestrates all three into a single lean prompt.

Common Myths

"Every agent needs all three memory tiers."
Not at all. A simple Q&A bot needs zero memory. A single-session agent might only need working memory. Episodic memory matters when users return across sessions. Procedural memory matters when the agent repeats complex workflows. Start with the tier that solves your actual problem.

"Episodic memory gives the agent perfect recall."
Episodic memory stores summaries, not transcripts. Summarization is lossy — specific names, numbers, and nuances may be dropped. If you need exact recall of a detail (like a contract amount), store it as structured metadata, not just in the summary text.

"More retrieved episodes = better responses."
Injecting 10 past episodes into the prompt does more harm than good. Each extra episode adds noise and tokens, and the model may confuse which details apply to the current task. Best practice: retrieve 2-3 most relevant episodes, max.

Remember This

Separate memory by purpose, not by storage.

The key insight of multi-layer memory is that **different kinds of information need different retrieval strategies and retention policies**. Working memory is fast and ephemeral (cleared per task). Episodic memory is persistent and searched by semantic similarity. Procedural memory is persistent and matched by task type.

A well-designed memory architecture keeps each LLM call lean — only the current state, the 2-3 most relevant past episodes, and a matching procedure template. That is typically under 4,000 tokens of memory context, compared to 50,000+ tokens if you tried stuffing raw conversation history into the prompt.

Production Gotcha

ChromaDB's default in-memory mode loses all data on process restart. Always use `PersistentClient(path="./chroma_db")` so episodic memory survives restarts, crashes, and deployments.

Test Yourself

1. The agent needs to store that the user prefers CSV output, learned last week. Which memory tier?

Answer: Episodic memory. User preferences from past conversations persist across sessions and are retrieved via semantic search when relevant. Working memory is for the current task only, and procedural memory stores action sequences, not preferences.

2. Why store conversation summaries instead of full transcripts in episodic memory?

Answer: Summaries are 10-20x smaller, produce better embeddings, and cost less to inject into prompts. A 4,000-token conversation compresses to ~300 tokens (93% reduction). Focused summaries also match more accurately during semantic search than rambling full transcripts.

3. An agent forgets user preferences between sessions despite using a vector database. Most likely cause?

Answer: ChromaDB is running in default in-memory mode. Without persistent storage configured, all stored episodes are lost when the process restarts. The fix: use `PersistentClient(path="./chroma_db")` so data writes to disk.

The ReAct Pattern: Reason, Act, Observe

The core loop that turns an LLM into an autonomous agent.

Interleaving Thinking with Doing

ReAct (Reasoning + Acting) is the pattern where Claude interleaves step-by-step thinking with tool execution in a loop. Each cycle has three phases:

1. **REASON** — Claude generates text explaining what it knows so far and what to do next.
2. **ACT** — Claude generates a `tool_use` block specifying the tool name and parameters.
3. **OBSERVE** — Your code executes the tool and sends results back as a `tool_result` message.

The loop repeats until Claude returns `stop_reason: "end_turn"` with no tool calls.

> Claude's Messages API is **natively ReAct** — no framework needed. Text blocks = Reason. `tool_use` blocks = Act. `tool_result` messages = Observe.

The Methodical Detective

> An “action-only” agent is like a detective who charges ahead — grabs the first suspect, skips the crime scene, ignores witnesses. They might solve easy cases by luck, but complex investigations turn into dead ends because critical evidence was never examined.

> Without stopping to reason, the agent makes locally reasonable but globally suboptimal decisions. It wastes tool calls on irrelevant searches, picks the wrong tool for the job, and cannot recover when it heads down the wrong path — because it never paused to evaluate whether the path was right.

> The ReAct pattern gives us a methodical detective. Before following any lead, they stop and **REASON**: “Based on the fingerprints and broken window, I should check the neighbor's alibi.” Then **ACT**: interview the neighbor. Then **OBSERVE**: “The alibi checks out.” With this new information, they reason again and choose a different lead. Each cycle builds on what was learned.

The ReAct Loop in 4 Steps

1. **User sends a question.** Your code wraps it in a messages array and calls Claude with a list of available tools. Claude receives both the question and the tool definitions.
2. **Claude responds with text + tool calls.** The response contains text blocks (reasoning) and `tool_use` blocks (actions). Check `stop_reason`: if **“tool_use”**, continue the loop. If **“end_turn”**, the agent is done.
3. **Your code executes the tool(s).** Run each requested tool with the parameters Claude specified, then append the results as `tool_result` messages to the conversation history.
4. **Loop back to step 2.** Claude reads the tool results, reasons about them, and decides whether to call another tool or deliver the final answer. The cycle repeats until Claude stops requesting tools.

The Agent Loop in Plain Code

No language-specific syntax — just the logic skeleton every ReAct agent follows.

```
FUNCTION run_agent(question):
  messages = [{role: "user", content: question}]

  LOOP (max 10 iterations):
    response = CALL Claude with tools + messages

    IF response.stop_reason == "end_turn":
      RETURN response.text   // done!

    IF response.stop_reason == "tool_use":
      APPEND assistant response to messages
      FOR each tool_use in response:
        result = EXECUTE tool(name, input)
        APPEND tool_result to messages
      CONTINUE loop
```

Three things your code *must* supply: the **tool list**, the **message history**, and the **stop conditions** (max iterations, token budget, timeout). Claude supplies everything else — the reasoning, tool selection, and final answer.

Three Things People Get Wrong

"An agent is just a chatbot with tools"

A chatbot with tools still needs a *human* to drive each step. An agent drives itself in an **autonomous loop**. The loop is the defining feature — Claude decides what to call next without waiting for another user message.

"ReAct requires a framework like LangChain"

No. Claude's Messages API naturally produces the ReAct pattern. You only need a `while` loop and tool execution logic. Frameworks add abstraction, but the pattern works with zero dependencies beyond the Anthropic SDK.

"The reasoning step wastes tokens — skip it"

Suppressing reasoning causes worse tool-selection decisions and destroys debuggability. You cannot trace *why* the agent chose a tool if it never explained its thinking. The tokens spent on reasoning are an investment in accuracy and observability.

What to Remember

ReAct = Reason + Act + Observe in a Loop

Claude's API does this natively — text blocks are reasoning, `tool_use` blocks are actions, `tool_result` messages are observations. You just write the loop and the tool handlers.

No framework required. Research shows ReAct improves task success rates by 20–30% over action-only approaches because the reasoning step helps the agent select better tools, recover from errors, and avoid wasted calls.

Your three engineering responsibilities:

1. **Define the tools** — what the agent can do.
2. **Write the loop** — call Claude, execute tools, append results.
3. **Set stop conditions** — max iterations, token budget, and timeout to prevent runaway costs.

Every advanced pattern in this course — planning, multi-agent systems, memory — is built on top of this loop.

Check Your Understanding

Tap “Reveal Answer” after thinking through each question.

Q1: What is the key difference between a chatbot and an agent?

Answer: An agent operates in an **autonomous loop**, deciding its own next actions, while a chatbot responds to individual prompts requiring human direction at every step. The loop — not the tools — is what makes it an agent.

Q2: How does your code know the agent is finished?

Answer: Check `response.stop_reason`. When it equals **“end_turn”**, Claude is done and the text contains the final answer. When it equals **“tool_use”**, continue the loop by executing the requested tools.

Q3: Why does Claude's API naturally implement ReAct without a framework?

Answer: Claude interleaves **text blocks** (reasoning) with **tool_use blocks** (actions), and **tool_result messages** serve as observations — matching the Reason-Act-Observe structure built directly into the API message format. No wrapper library needed.

Planning & Task Decomposition

Teach your agent to think before it acts — break complex goals into structured plans.

Track 4: Architecture
Module 13 of 30

Planning & Task Decomposition

Teach your agent to think before it acts — break complex goals into structured plans.

Why Agents Need a Planning Layer

A raw ReAct agent chains tool calls one step at a time. For tasks with **5+ steps**, it loses track of the overall goal, makes locally reasonable but globally suboptimal decisions, and misses dependencies. Research shows ReAct-only agents complete complex tasks **~40% of the time**; adding a planning layer pushes that to **~80%**.

Planning is a **meta-strategy**: before making any tool calls, the agent analyzes the goal and produces a structured execution plan — a DAG (Directed Acyclic Graph) of sub-tasks with dependencies. Planning answers *"what to do and in what order"*; ReAct handles *"how to execute each step."*

The full pipeline: **Intent Classification → Task Decomposition → DAG Execution → Final Synthesis.** Each phase has a single responsibility, and together they transform a reactive tool-caller into a strategic problem-solver that completes complex goals reliably.

IKEA Furniture Without Instructions

> Building IKEA furniture without reading the instructions. You open the box, see 47 pieces and a bag of screws, and just start connecting things that look like they fit. Each individual connection seems reasonable in the moment.

> You attach a side panel upside down, realize at step 15 you missed a bracket at step 3, and end up with leftover screws you're not sure about. Backtracking wastes time and sometimes requires disassembly — undoing work you already did just to fix the foundation.

> Planning is reading the instructions first. The agent analyzes the entire goal, identifies phases, maps dependencies, and creates a structured plan. Independent steps can run in parallel. This transforms "try things and hope" into "understand, plan, execute systematically."

Planning Pipeline in 5 Steps

1. **Intent Classification.** A quick check: does this request even need planning? Simple questions get direct answers. Single-tool tasks skip straight to execution. Only multi-step tasks enter the planning pipeline.
2. **Task Decomposition.** Claude breaks the complex goal into a DAG of sub-tasks. Each sub-task has an ID, description, dependencies, and suggested tools.
3. **Dependency Validation.** Run topological sort to ensure no circular dependencies. If a cycle is detected, fall back to a sequential single-task plan.
4. **DAG Execution.** Find all tasks with no unfinished dependencies ("ready tasks") and run them in parallel. As each completes, check for newly unblocked tasks. Repeat until all tasks are done.
5. **Synthesis.** Gather all sub-task results and have Claude produce a final unified answer that addresses the original goal holistically.

Plan-and-Execute Pattern

```
FUNCTION plan_and_execute(user_goal):
  intent = CLASSIFY(user_goal)
  IF intent.needs_planning == false:
    RETURN simple_answer(user_goal)

  tasks = DECOMPOSE(user_goal)  // DAG of sub-tasks
  VALIDATE no circular dependencies

  results = {}
  WHILE unfinished tasks exist:
    ready = tasks WHERE all dependencies DONE
    RUN ready tasks IN PARALLEL
    STORE results for each completed task

  final = SYNTHESIZE(all results, user_goal)
  RETURN final
```

The key insight: **CLASSIFY** acts as a gate — simple requests skip planning entirely. **DECOMPOSE** produces the DAG structure with explicit dependencies. The **WHILE** loop finds parallelizable work at each iteration, and **SYNTHESIZE** merges all sub-task results into one coherent response.

Common Myths

> **"Every request needs a plan."**
> Simple questions and single-tool tasks don't benefit from planning. The extra API call to generate a plan makes them slower. Intent classification exists to skip planning when it's not needed.

> **"The plan must be perfect before execution starts."**
> Plans are hypotheses, not contracts. A good agent executes but adapts when sub-tasks return unexpected results. Rigid plan-following is as bad as no planning.

> **"More intent categories = better routing."**
> More categories increase misclassification risk. Start with 3–4 broad intents. Split only when real data shows failures.

Remember This

> **Plan before you act, but adapt as you go.**
>
> Planning transforms agents from reactive tool-callers into strategic problem-solvers. The key pattern: **classify intent first** (skip planning for simple tasks), **decompose** complex goals into a DAG of sub-tasks, **execute** independent tasks in parallel, and **synthesize** results.
>
> Always validate the DAG for cycles, and default to **"needs planning"** when unsure — over-planning wastes a few hundred tokens, under-planning causes task failure. The asymmetry is clear: when in doubt, plan.

Test Yourself

1. Why does a ReAct-only agent struggle with 5+ step tasks?

Answer: Without a plan, it loses track of the overall goal, makes locally reasonable but globally suboptimal choices, and misses dependencies between steps.

2. What is a DAG and why must it be acyclic?

Answer: A Directed Acyclic Graph where nodes are tasks and edges are dependencies. Cycles create deadlocks — if A depends on B and B depends on A, neither can ever start.

3. Why default to needs_planning=true when classification is uncertain?

Answer: False positives (simple task goes through planning) waste a few hundred tokens. False negatives (complex task skips planning) cause failure and expensive retries. Over-planning is cheap; under-planning is costly.

Multi-Agent Systems

Split complex work across specialized agents that collaborate through a coordinator.

Track 4: Architecture
Module 14 of 30

Multi-Agent Systems

Split complex work across specialized agents that collaborate through a coordinator.

Specialized Agents Beat Generalists

A multi-agent system decomposes complex tasks across multiple specialized LLM-powered agents. Each agent has a **focused system prompt** (200–400 tokens vs 1,000+ for a generalist), a **small relevant tool set** (3–5 tools, not 15+), and **bounded responsibilities**.

A coordinator agent orchestrates delegation and aggregates results. Use multi-agent only when sub-tasks require fundamentally different expertise, tools, and reasoning styles.

- **Prompt specialization** — shorter prompts are followed more reliably
- **Tool isolation** — accuracy degrades above 5 tools per agent
- **Parallel execution** — independent agents can run concurrently
- **Failure isolation** — one agent crashing does not take down the entire pipeline

The Construction Crew

> A solo contractor asked to build a skyscraper — architect, structural engineer, electrician, plumber, and project manager all at once. Even with all those skills, they would be overwhelmed by constant context switching between fundamentally different domains.

> They cannot work on the foundation and the wiring simultaneously. Every tool switch requires mentally reloading an entire domain's rules. Quality suffers across the board because no single prompt can deeply specialize in everything at once.

> The multi-agent system is a construction crew. The **Researcher** has search tools and a "gather facts" prompt. The **Writer** has a "produce prose" prompt and no tools. The **Editor** focuses on grammar and clarity. The **Supervisor** coordinates, just like a project manager. Each specialist handles what they are best at.

The Multi-Agent Pipeline

1. **RECEIVE** — The coordinator receives the user's request and analyzes what types of work are needed: research, writing, editing, review, or other specialized tasks.
2. **DELEGATE** — The coordinator delegates to specialist agents via structured handoff messages containing the original goal, specific instructions, and any prior results from other agents.
3. **EXECUTE** — Each specialist runs its own ReAct loop with its focused tools and prompt, returning structured results back to the coordinator when finished.
4. **AGGREGATE** — The coordinator aggregates results. If a reviewer rejects quality, the coordinator loops back to the writer with feedback. Once approved, it delivers the final output to the user.

Content Pipeline Example

```
FUNCTION content_pipeline(topic):
  research = DELEGATE to Researcher:
    "Gather facts about {topic}"

  LOOP (max 3 attempts):
    draft = DELEGATE to Writer:
      "Write article using {research}"
    edited = DELEGATE to Editor:
      "Polish {draft} for clarity"
    review = DELEGATE to Reviewer:
      "Score and approve/reject {edited}"

    IF review.approved: RETURN edited
    ELSE: inject review.feedback
          into next attempt

  RETURN best draft so far
```

The write–edit–review loop with feedback injection is the core quality pattern. Each specialist operates in its own context window with a focused prompt and small tool set. The coordinator manages the flow and decides when quality is sufficient.

Common Myths

"More agents = better results."
Each agent adds coordination overhead — extra API calls, message formatting, and result aggregation. Two agents for a task one can handle just doubles the cost. Use multi-agent only when sub-tasks genuinely need different expertise.

"Agents are isolated because they are separate API calls."
They are isolated in context windows, but they share side effects. If Agent A writes to a database and Agent B reads from it, they are implicitly coupled. Shared-state coordination must be explicit and carefully managed.

"The supervisor needs to be the smartest model."
The supervisor mostly routes and aggregates. A cheaper, faster model (Haiku) often works well for coordination. Save the powerful model (Sonnet/Opus) for the workers doing the heavy reasoning and content generation.

Remember This

> **Hub-and-spoke is the go-to multi-agent pattern.**
>
> Multi-agent systems split complex tasks across focused specialists coordinated by a supervisor. The recommended architecture is **hub-and-spoke**: one coordinator delegating to workers, each with a small tool set and focused prompt.
>
> The write–edit–review loop with feedback injection is the core quality pattern. Only use multi-agent when sub-tasks genuinely need different expertise — a single planning agent is simpler and cheaper when all sub-tasks share the same reasoning style.
>
> - Each specialist gets **3–5 tools** and a **200–400 token** system prompt
> - The coordinator handles **routing, aggregation, and quality gates**
> - Feedback loops prevent quality degradation across handoffs

Test Yourself

1. When should you use multi-agent instead of a single planning agent?

Answer: When sub-tasks require fundamentally different expertise, tools, and reasoning styles. If all sub-tasks use the same reasoning, a single planning agent is simpler and cheaper. Multi-agent adds coordination overhead that only pays off when specialists genuinely outperform a generalist.

2. Which architecture pattern is recommended for most multi-agent use cases?

Answer: Hub-and-spoke (supervisor/worker) — a single coordination point with clear context isolation between workers and structured result aggregation. The supervisor delegates tasks, collects results, and manages quality gates without doing the heavy reasoning itself.

3. What must a handoff message include to prevent context loss?

Answer: Sender, receiver, task ID, message type, payload, the original user goal, specific instructions, and metadata. The original goal is critical — downstream agents need to know the user's actual intent, not just the immediate sub-task, to make good decisions when ambiguity arises.

Code Interpreter & Sandbox Execution

Give your agent the power to write and run real code — safely.

Track 4: Architecture
Module 15 of 30

Code Interpreter & Sandbox Execution

Give your agent the power to write and run real code — safely.

Why Agents Need a Real Interpreter

LLMs are probabilistic text generators — they approximate answers rather than computing them deterministically. A **code execution tool** bridges this gap: Claude writes the code, a sandboxed interpreter runs it, and the exact result comes back.

Claude with code execution scores **94% on math and data tasks** vs 67% without — a 27-point improvement. For financial analysis: **97% vs 52%**. The division of labor is clear: Claude handles understanding, algorithm choice, and interpretation. The interpreter handles precise, deterministic execution.

The sandbox ensures untrusted code can never affect your host system. No file access, no network calls, no lingering processes. The container is created, used once, and destroyed — a disposable computation engine.

The Mathematician & the Calculator

> A brilliant mathematician who can explain calculus beautifully but occasionally makes arithmetic errors when doing long division by hand. They understand the concept perfectly — they just sometimes fumble the computation.

> When your agent reports “the churn rate is approximately 3.7%,” stakeholders can’t trust the number. Was it 3.663% or 3.742%? In finance and healthcare, approximate answers can mean compliance violations or wrong decisions.

> Give the mathematician a calculator. They still do the thinking — “I need to divide churned customers by total customers and multiply by 100” — but delegate the computation to a tool that never makes arithmetic mistakes. The sandbox is a disposable kitchen: fully equipped but in a separate building, demolished after use.

The 4-Step Execution Loop

1. **RECEIVE** — Claude receives a question requiring computation (e.g., “What’s the churn rate for Q3?”). It reasons about the approach and writes Python code as a tool call.
2. **SANDBOX** — Your code intercepts the tool_use block, extracts the code string, and launches a Docker container with security flags: no network, memory limit, CPU limit, read-only filesystem.
3. **EXECUTE** — The container runs the code and returns stdout, stderr, exit_code, and execution_time. The container is destroyed immediately after.
4. **RETURN** — If the code succeeded, results go back to Claude as a tool_result. If it failed, Claude reads the error and rewrites the code (self-debugging, up to 3 retries). After 3 failures, Claude falls back to its best answer without code.

The Sandbox Execution Pattern

```
DEFINE tool "run_python"
  input: code (string), timeout (default 30s)

FUNCTION execute_sandboxed(code):
  container = DOCKER RUN with:
    no-network, memory=256MB, read-only
  result = RUN code in container
  DESTROY container
  RETURN {stdout, stderr, exit_code, time}

FUNCTION agent_loop(question):
  LOOP (max 10 turns, max 3 code retries):
    response = ASK Claude with run_python tool
    IF done: RETURN response.text
    IF tool_use: run execute_sandboxed(code)
    IF code failed 3 times: ask Claude for
      best answer without code
```

The sandbox is created fresh, used once, and destroyed. There is no state between executions — each run starts clean. If code errors, Claude reads stderr, reasons about the fix, and retries automatically up to three times before falling back gracefully.

Common Myths

"Code execution means the LLM runs code itself."
The LLM generates code as text. A separate interpreter executes it. The LLM never "runs" anything — it only predicts the next token. Your orchestrator bridges the gap between generated text and real execution.

"Docker is overkill — subprocess with a timeout is enough."
A timeout only prevents infinite loops. Without Docker, code runs with your server's permissions and can read SSH keys, environment variables, and the entire filesystem. Sandboxing is about isolation, not just time limits.

"More retries = better results."
After 3 attempts, the issue is usually systemic (wrong approach, missing package). Each retry costs one API call + one sandbox execution. Diminishing returns hit fast — cap retries at 3 to avoid wasting money on unfixable problems.

Remember This

Code execution transforms approximate into precise.

The pattern is simple: Claude writes code, a sandboxed interpreter runs it, results come back. Always sandbox with Docker: **no network**, **memory limits**, **read-only filesystem**, **timeout**.

The self-debugging loop (feed errors back to Claude) fixes ~80% of first-attempt failures, but cap retries at 3 to avoid wasting money on unfixable problems. Adding code execution is one of the highest-leverage single changes you can make — 94% vs 67% accuracy on math tasks is the difference between a trustworthy tool and a liability.

Test Yourself

1. Why can't LLMs reliably perform precise calculations without code execution?

Answer: LLMs are **token predictors** — they generate the most likely next token based on patterns, which works for language but fails for precise arithmetic. Code delegates computation to a deterministic interpreter that returns exact results every time.

2. Your Docker sandbox uses: docker run --rm --memory=256m python:3.12. What security vulnerability exists?

Answer: Missing **--network=none**. Without it, generated code can make outbound network calls to exfiltrate API keys, user data, or system information. Always disable network access unless the task explicitly requires it.

3. Agent code fails with "ModuleNotFoundError: No module named scipy". What should the self-debugging loop do?

Answer: Feed the error back to Claude, which **rewrites the code using available alternatives** (e.g., numpy or the statistics module instead of scipy). This pattern succeeds ~80% of the time without requiring any package installation in the sandbox.

Build a Complete Agent & Subagent System

BUILD LAB

Your Tools Are Other Agents

This BUILD lab teaches you to assemble a complete multi-agent system using the **hub-and-spoke architecture**. You build a coordinator agent whose "tools" are not database queries — they are other agents.

The coordinator receives user queries and delegates to specialists: a **research subagent** that searches UCC filings, and an **analysis subagent** that computes risk scores.

The central insight: **each subagent starts with a blank context window**. It only knows what the coordinator explicitly passes. Context doesn't flow automatically — you must build the bridge.

The lab starts by building a single-agent version first, then refactors into multi-agent so you see exactly when and why the split helps. This progression is deliberate: you should always prove a single agent can't handle the complexity before splitting into multiple agents.

The Call Center

> A single call center operator answering every question — billing, tech support, returns, scheduling. One person knowing everything, callers waiting while they look things up across five different systems.

> The operator context-switches constantly, gives shallow answers because they can't deeply specialize, and the queue grows because only one person handles everything. Sound familiar? This is your single agent with too many tools and responsibilities.

> Modern call centers route you to a specialist. "Press 1 for billing, press 2 for technical support." Each specialist knows their domain deeply and has the right tools on their screen. A supervisor can conference in multiple specialists for complex cases. Your agent system mirrors this: a coordinator reads the question and routes to the right subagent.

The 5-Step Pipeline

1. **User asks a question** (e.g., "What's the lien risk for Acme Corp?"). The coordinator agent receives it.
2. **Coordinator decides which subagent(s) to invoke** — it calls its "meta-tools" (`research_filings`, `analyze_risk`), which are actually other agents.
3. **Research subagent runs its own ReAct loop** with `search_filings` and `get_filing_details` tools. It returns structured JSON with findings.
4. **Coordinator extracts key data** from research results and builds a new task string for the analysis subagent, explicitly including debtor ID and filing summary.
5. **Analysis subagent computes a risk score** and returns it. The coordinator synthesizes both results into a final answer for the user.

The System in Plain Logic

No language-specific syntax — the architecture in pure logic.

```
FUNCTION coordinator(user_question):
  response = ASK Claude with meta-tools:
    [research_filings, analyze_risk]

  WHEN tool "research_filings" called:
    findings = run_research_subagent(task)
    // subagent has: search_filings, get_details
    RETURN findings as JSON

  WHEN tool "analyze_risk" called:
    // MUST pass context explicitly!
    task = "Analyze {debtor_id}, {filing_count}
            filings totaling {amount}"
    score = run_analysis_subagent(task)
    RETURN score as JSON

  RETURN coordinator's final synthesis
```

Notice the comment on line 10: the coordinator **must** explicitly pass context. The analysis subagent has a blank context window — it cannot see what the research subagent found unless the coordinator includes it in the task string.

Three Things People Get Wrong

"Subagents share the coordinator's memory"

Each subagent starts with a completely empty context window. It only knows what the coordinator explicitly passes in the task description. If the coordinator forgets to include the debtor_id, the subagent cannot look it up — it simply doesn't have that information.

"More subagents = better system"

The coordinator treats subagents as tools. Just like tool selection degrades above 5 tools, coordinator routing degrades above 5 subagents. If you need more, use hierarchical coordination — intermediate coordinators managing smaller groups of specialists.

"I should use multi-agent for everything now"

A single agent with 2–3 tools is simpler, cheaper, and often good enough. Multi-agent pays off with 5+ tools, context isolation needs, or separation of concerns. The lab builds single-agent first to prove this point deliberately.

What to Remember

The Coordinator's Tools ARE Other Agents

That's the key architectural insight of this entire module. Each subagent gets its own focused prompt, narrow tool set, and blank context window.

The design principles that make this work:

- **Context must be explicitly passed** between agents — nothing flows automatically from coordinator to subagent.
- **Always return structured JSON** from subagents so results can be programmatically chained and parsed by the coordinator.
- **Start with a single agent** and split into multi-agent only when complexity demands it — more agents means more coordination overhead.
- **Each subagent owns its domain completely** — focused prompt, focused tools, focused output format.

Check Your Understanding

Think through each question, then tap to reveal the answer.

Q1: How does the coordinator pass context to the analysis subagent?

Answer: Explicitly in the **task description string**, including the debtor_id and filing summary extracted from the research results. Subagents have isolated context windows and do **not** inherit the coordinator's conversation history. If the coordinator doesn't include a piece of data, the subagent simply doesn't have it.

Q2: What does stop_reason "tool_use" mean in the agent loop?

Answer: It means Claude wants to call a tool — execute the requested tool and continue the loop. When stop_reason changes to **"end_turn,"** Claude is done and has produced its final response. The agent loop keeps running as long as it sees "tool_use."

Q3: You have a coordinator with 15 subagents. What is the problem?

Answer: Same problem as too many tools — the coordinator cannot reliably pick which subagent(s) to call when there are 15 options. **Solution:** hierarchical coordination with intermediate coordinators, each managing a smaller group of 3–5 specialized subagents.

Input Guardrails

Build the first line of defense: validate, sanitize, and protect every input before it reaches your agent.

Track 5: Guardrails & Safety
Module 18 of 30

Input Guardrails

Build the first line of defense: validate, sanitize, and protect every input before it reaches your agent.

Defense in Depth for Every Input

Input guardrails are validation layers that sit between the user and the agent's core reasoning. Every incoming message is intercepted and run through a series of checks **before it ever reaches Claude**.

The architecture follows **defense in depth** — multiple concentric layers, each catching different failure modes. The pipeline runs cheapest checks first:

- **Rate Limiter** — token bucket per user, zero latency
- **PII Detector** — regex scan for SSNs, credit cards, emails
- **Injection Classifier** — separate Claude call to detect prompt hijacking
- **Schema Validator** — structure, types, value bounds
- **Claude** — clean input finally reaches the model

If any layer blocks, the pipeline **short-circuits** and the Claude API call is never made. Three possible outcomes: **PASS** (clean input), **MODIFIED** (PII redacted but intent legitimate), or **BLOCKED** (injection attempt or rate limit exceeded).

Highway Guardrails

> A highway on a sunny day with light traffic — everything works perfectly fine. No guardrails needed when conditions are ideal. Every car stays in its lane, every driver is alert, and the road seems completely safe without any barriers.

> The first unexpected curve, the first patch of ice, the first distracted driver — and suddenly a car goes off a cliff. The road itself didn't break; nothing existed to absorb mistakes and prevent catastrophe. Your agent works beautifully with well-formed test inputs, but in production users send malformed JSON, include SSNs in casual chat, and try to hijack your system prompt with clever injection attacks.

> Highway guardrails keep everything on the road. Input guardrails intercept every message before it reaches Claude. PII regex redacts sensitive numbers. A separate Claude call classifies injection attempts. Schema validation catches malformed requests. Rate limiting prevents cost explosions. Each layer catches what the others miss — defense in depth.

The 5-Layer Input Pipeline

1. **RATE LIMITER** — checks first with zero latency. Each user gets a token bucket that refills over time. If the bucket is empty, the request is rejected immediately — no further processing happens.
2. **PII DETECTOR** — scans for structured patterns: SSNs, credit card numbers (with Luhn checksum validation), emails, and phone numbers. Matches are replaced with typed tokens like `[REDACTED_SSN]`. The input continues with redacted text.
3. **INJECTION CLASSIFIER** — makes a separate, lightweight Claude call with its own clean context. Classifies input as safe, suspicious, or malicious. If malicious, the pipeline blocks. Critical rule: **fail closed** — if the classifier itself errors, block the input.
4. **SCHEMA VALIDATOR** — checks structure: required fields present, types correct, values within bounds (e.g., max_tokens capped at 4096), tool names against an allowlist. Zero API calls, instant validation.
5. **CLEAN INPUT REACHES CLAUDE** — if PII was found and redacted, the status is MODIFIED. If everything was clean from the start, status is PASS. Either way, only sanitized input reaches the model.

Input Pipeline Overview

```
FUNCTION process_input(raw_input, user_id):
  // Layer 1: Rate limit (cheapest first)
  IF NOT rate_limiter.allow(user_id):
    RETURN blocked("rate_limit_exceeded")

  // Layer 2: PII redaction
  clean_text, pii_found = redact_pii(raw_input)

  // Layer 3: Injection detection
  threat = classify_injection(clean_text)
  IF threat == "malicious":
    RETURN blocked("injection_detected")

  // Layer 4: Schema validation
  VALIDATE clean_text against schema

  RETURN {status: pii_found ? "modified" : "pass",
          sanitized: clean_text}
```

The pipeline runs cheapest checks first. Rate limiting and schema validation cost nothing — just local logic. PII regex is sub-millisecond. Only the injection classifier requires an API call (~200ms). If any early layer blocks, you never pay for the expensive checks downstream.

Common Myths

"The system prompt says 'don't reveal instructions' — that's enough."
System prompt instructions are advisory, not enforced. A clever injection can override them. You need **programmatic guardrails** that run BEFORE the input reaches the model — code that the user's text cannot influence.

"If the guardrail crashes, let inputs through."
This is "failing open" — a critical security mistake. If an attacker can crash the classifier, they've disabled all guardrails. Security systems must **fail closed**: if uncertain, block. It's better to reject a legitimate input than to let an attack through.

"Regex catches all PII."
Regex catches **structured PII** — SSNs, credit card numbers, email addresses. It completely misses unstructured PII like names and addresses. "John Smith lives at 42 Oak Lane" has three PII items no regex will catch. You need NER models or LLM-based detection for unstructured data.

Remember This

Input guardrails are layered defenses that run before Claude sees any input.

Order them **cheapest-first**: rate limiting (free), PII regex (sub-millisecond), injection classification (~200ms API call), schema validation (free). This ordering minimizes cost and latency — most blocked requests never trigger an API call.

Always fail closed. If any guardrail errors or returns an uncertain result, block the input rather than letting it through. A false positive is annoying; a false negative is a security breach.

The injection classifier **must use a separate Claude call** with its own clean context. If the classifier shares context with the main agent, the injection attempt is already in the context and could manipulate the classifier's judgment.

Test Yourself

1. What is the key difference between direct and indirect prompt injection?

Answer: Direct injection comes from the user's own message — the user deliberately tries to hijack the agent. **Indirect injection** comes from external data the agent retrieves — a poisoned web page, a compromised RAG document, or a malicious database record that the agent trusts as clean input.

2. Why must the injection classifier use a SEPARATE Claude call?

Answer: If the classifier shares context with the main agent, **the injection attempt is already in the context** and could fool the classifier. A separate call has a clean, purpose-built context that the malicious input cannot manipulate — the classifier judges the input from the outside, not from within the compromised conversation.

3. A token bucket has capacity=10 and refill_rate=2/sec. A user sends 100 requests in 10 seconds. How many are accepted?

Answer: About 30. The initial 10 come from the full bucket. Over the next 10 seconds, the bucket refills at 2 tokens per second, yielding ~20 more accepted requests. The remaining ~70 requests are rejected. Token buckets allow bursts up to capacity while enforcing a sustained rate limit.

Output Guardrails & Human-in-the-Loop

Validate what your agent says, control what it costs, and know when to hand off to a human.

Track 5: Guardrails & Safety
Module 19 of 30

Output Guardrails & Human-in-the-Loop

Validate what your agent says, control what it costs, and know when to hand off to a human.

Protecting Users FROM Your System

Input guardrails (M16) protect your system from bad user inputs. **Output guardrails protect users from your system's mistakes.** They sit between the agent's response and the user, catching problems before they cause harm.

Four critical layers run on every response:

- **Hallucination detection** — fact-check AI outputs against source documents. The model can confidently state completely wrong things.
- **Toxicity filtering** — block harmful content that may leak in from external data sources during agentic loops.
- **Format validation** — ensure responses match expected schemas: valid JSON, required fields, correct data types.
- **Cost controls** — prevent runaway agent loops from burning your budget in minutes.

Beyond individual checks, **HITL (Human-in-the-Loop) gates** pause at high-stakes decision points for human review, and **circuit breakers** prevent cascading failures when downstream systems fail.

Quality Inspector on the Assembly Line

> Products go straight from machines to customers. Most are fine, but occasionally a defective one slips through — a faulty brake line, a sharp edge, a wrong dosage. The factory looks efficient, but it's gambling with every product.

> The machines don't know they made a mistake. There's no checkpoint to catch defects before products reach real people. Your agent is the same: it generates responses with full confidence, even when those responses contain fabricated facts or malformed data.

> Quality inspectors sit at the **end** of the assembly line, checking every product before it ships. Output guardrails work identically — catching hallucinations, toxic content, and malformed output before the response reaches your user.

The 5-Step Output Pipeline

1. **FORMAT CHECK** — Validate output structure: valid JSON? Required fields present? Values in range? This is the cheapest check — no API call needed, just local parsing.
2. **HALLUCINATION DETECTION** — A separate Claude call compares response claims against source documents. Each claim is classified as supported, unsupported, or contradicted. Contradicted claims trigger a block.
3. **TOXICITY FILTER** — Claude-as-judge rates content on a severity scale and blocks anything above your threshold. Catches harmful content leaked from tool outputs or RAG retrieval.
4. **COST CONTROLS** — Middleware tracks tokens consumed and cumulative cost. Per-request budget caps prevent runaway loops. If cost exceeds budget, the agent returns a safe fallback.
5. **HITL GATES** — Three patterns: **Approval** (human yes/no), **Modification** (human edits agent's draft), and **Escalation** (agent recognizes its own limits and routes to a human).

Hallucination Check Pipeline

```
FUNCTION checkHallucination(response, sources):
  factCheck = ASK Claude "compare claims to sources"

  FOR EACH claim in factCheck:
    IF claim contradicts sources:
      MARK "contradicted" → BLOCK response
    ELSE IF claim has no source support:
      MARK "unsupported" → FLAG for review
    ELSE:
      MARK "supported" → PASS

  IF any contradicted: RETURN "block"
  ELSE IF 2+ unsupported: RETURN "flag"
  ELSE: RETURN "pass" → deliver to user
```

Format checks run first because they're free. No point spending $0.003 on a hallucination check if the JSON is already broken. Cost checks act as circuit breakers, killing runaway loops before they drain your budget.

Common Myths

"Output guardrails are unnecessary because Claude is already safe."
Claude's safety training catches most harmful content, but not hallucinations. The model can confidently state completely incorrect facts. In agentic loops, external data from tool calls can leak offensive content into responses the model's training never anticipated.

"The model can tell you how confident it is."
Self-reported confidence ("I'm 85% confident") is **not calibrated**. Claude might express high confidence about a hallucinated fact. Use programmatic checks — source document matching, structured validation — not the model's self-assessment.

"Human-in-the-loop means the AI is unreliable."
The opposite is true. Even the best employees need manager approval for large transactions. HITL gates are about **risk management**, not reliability. The agent does 95% of the work; the human provides judgment on decisions that matter most.

Remember This

> **Output guardrails are the last checkpoint before your agent's decisions affect real people.**
>
> **Format validation is free** — just local JSON parsing. **Hallucination detection costs ~$0.003 per check** — trivially cheap compared to acting on fabricated data.
>
> **Cost controls** prevent one bad recursive loop from burning your entire monthly budget. **HITL gates** ensure humans handle irreversible actions, policy edge cases, and capability limits the agent recognizes in itself.
>
> Together with input guardrails (M16), output guardrails complete the safety sandwich: protect your system from users *and* protect users from your system.

Test Yourself

1. What is the key difference between an approval gate and a modification gate?

Answer: Approval gates are binary (yes/no) — the human either approves or rejects the agent's action. **Modification gates allow the human to edit the content** before it proceeds. Use approval for simple go/no-go decisions; use modification when the agent's draft is close but needs tweaking.

2. Why shouldn't you trust Claude's self-reported confidence scores?

Answer: They're not calibrated. Claude can say "very confident" about a hallucinated fact. Use programmatic verification — compare claims against source documents, validate structure locally — instead of asking the model how sure it is.

3. An agent claims "filed on March 15" but sources show March 22. Which guardrail catches this?

Answer: Hallucination detector. The claim directly contradicts the source documents and should be blocked. Toxicity filters catch harmful content, not factual errors. Format validators check structure, not semantic accuracy.

Evaluation & Testing

Measure agent quality with rubric-based scoring, not pass/fail assertions.

Track 5: Guardrails & Safety
Module 20 of 30

Evaluation & Testing

Measure agent quality with rubric-based scoring, not pass/fail assertions.

You Can't Assert Exact Outputs

Agent evaluation is fundamentally different from traditional software testing. LLM outputs are **non-deterministic** — the same input can produce different valid outputs every time. You cannot write `assert response == "exact string"`.

Instead, you measure quality using **rubric-based evaluation** across multiple dimensions: task completion, tool accuracy, response quality, and efficiency. A single number like "87% accuracy" hides critical information.

An agent might complete tasks correctly but waste tokens inefficiently, or call the wrong tools despite getting the right answer. **Multi-dimensional metrics with Claude-as-judge** reveal the full picture, letting you make informed trade-offs before deploying new versions.

Restaurant Health Inspection

> A chef changes a recipe — swaps one ingredient, adjusts cooking time — without anyone checking whether the food is still safe. Maybe the new seasoning is great, but the lower temperature means the chicken isn't fully cooked. Nobody notices until customers get sick.

> Without systematic checks, a small change can silently break things. A 5-word edit to your system prompt could degrade 20% of edge cases. You won't know until support tickets spike days later.

> Health inspections run the **same checklist every time**, regardless of what changed. Agent regression testing works identically: every time you change a prompt, add a tool, or update logic, you re-run the same evaluation suite. It catches regressions in 5 minutes instead of in production.

The 5-Step Evaluation Cycle

1. **BUILD TEST SUITE** — 50 curated test cases covering common scenarios (60%), edge cases (25%), and adversarial inputs (15%). Each has an input, expected behavior description, category tags, and difficulty level.
2. **RUN AGENT** — Execute the agent on each test case, capturing the full output, token count, and every tool call made. Store these outputs for scoring.
3. **SCORE WITH CLAUDE-AS-JUDGE** — A separate Claude call with a detailed rubric scores each output on task completion, tool accuracy, and response quality. The judge must provide reasoning BEFORE scoring — this dramatically improves consistency.
4. **COMPUTE METRICS** — Calculate aggregate scores per dimension AND stratified metrics by category. The aggregate can hide category-specific failures.
5. **DETECT REGRESSIONS** — Compare new version against baseline. Flag >5% drops as warnings and >10% drops as critical blockers.

Evaluation Harness

```
DEFINE TestCase:
  input, expected_behavior, category, difficulty

FUNCTION evaluate(test_suite):
  FOR EACH test_case in test_suite:
    output = RUN agent with test_case.input
    scores = ASK Claude-as-judge with rubric:
      "Rate task_completion, tool_accuracy,
       response_quality on 1-5 scale.
       Provide reasoning BEFORE each score."
    STORE scores + reasoning

  COMPUTE aggregate: mean per dimension
  COMPUTE stratified: mean per category

FUNCTION compare(baseline, candidate):
  FOR EACH metric:
    delta = (candidate - baseline) / baseline
    IF delta < -10%: BLOCK deploy
    IF delta < -5%: WARN, investigate
    ELSE: SAFE
```

The judge prompt must be MORE specific than the agent prompt. Vague criteria like “evaluate helpfulness” produce scores varying by 1-2 points. Detailed rubrics reduce variance to ~0.3 points.

Common Myths

“I’ll set temperature to 0 for deterministic tests.”
Temperature 0 reduces variation but doesn’t eliminate it. Different hardware, SDK versions, or API batching can still produce different outputs. Build tests to **tolerate variation** with rubric-based scoring, not eliminate it.

“If the final answer is right, the test passes.”
An agent that calls 15 tools for a question that should take 2 is broken, even if the answer is correct. Evaluate the **entire trajectory** — tool calls, reasoning steps, decisions — not just the endpoint.

“I need thousands of test cases.”
Start with **50 well-curated cases** covering common scenarios, edge cases, and adversarial inputs. 50 high-quality cases beat 1,000 random ones. Quality (representative coverage) matters more than quantity.

Remember This

Agent evaluation requires multi-dimensional rubric-based scoring, not binary pass/fail.

Use **Claude-as-judge** with detailed rubrics to score each output on task completion, tool accuracy, and response quality. Require reasoning before scores for consistency.

Run the same **50-case evaluation suite on every code change** and compare metrics against baseline. This turns “I think this prompt is better” into data-driven decisions backed by stratified metrics and statistical significance.

Test Yourself

1. Why do exact-match assertions fail for agent testing?

Answer: LLMs are non-deterministic — identical inputs produce different (but valid) outputs due to probabilistic sampling. Use rubric-based evaluation with Claude-as-judge that scores quality on multiple dimensions instead.

2. Your v1.1 improves task completion by 4% but degrades response quality by 8%. Deploy?

Answer: Flag for investigation. The 8% quality drop exceeds the 5% warning threshold. You need stratified analysis to see WHERE quality dropped before deciding. The improvement in one dimension doesn't automatically justify regression in another.

3. Why must the judge prompt be MORE specific than the agent prompt?

Answer: Vague criteria produce inconsistent scores. "Evaluate helpfulness" gives scores varying by 1-2 points. A detailed rubric with explicit criteria per level ("5 = addresses all questions, actionable steps, professional tone") reduces variance to ~0.3 points.

Tracing & Logging

See inside your agent's decisions with traces, spans, and structured logs.

Track 6: Observability
Module 21 of 30

Tracing & Logging

See inside your agent's decisions with traces, spans, and structured logs.

Stop Debugging Blind

Observability is the ability to understand a system's internal state by examining its external outputs — traces, logs, and metrics. Unlike monitoring (which tells you *that* something broke), observability tells you **why** it broke.

AI agents are non-deterministic — the same input can produce different outputs. Without observability, when a user reports "the agent gave me the wrong answer," you have no trace of which tools were called, what the model reasoning was, or where the chain broke.

Observability captures **every execution as it happens**, giving you a permanent, queryable record to analyze after the fact — without deploying new code or reproducing unreproducible issues.

Driving at Night

> Running an AI agent without observability is like driving at night with no headlights, no dashboard gauges, and no GPS. You press the accelerator and hope you're going the right speed, in the right direction, on the right road.

> If something goes wrong, you have no way to know what happened until you crash. You can't reproduce the exact conditions, and you can't investigate what happened at mile 47 when you're already at mile 200.

> **Traces** = your GPS showing the road your agent took. **Spans** = timing for each turn and acceleration. **Structured logs** = your gauges — real-time readings of speed, fuel, and engine temperature. With these instruments, you see exactly what happened, when, and why.

5 Steps to Observability

1. **CREATE A TRACE** — When a user request arrives, initialize a unique trace_id that links all subsequent operations. The trace is the top-level container for the complete request lifecycle.
2. **NEST SPANS** — For each operation (LLM call, tool invocation, guardrail check), create a child span with timestamps, status, and metadata. Spans form a tree showing execution order.
3. **LOG STRUCTURED JSON** — Emit machine-parseable JSON with consistent keys: trace_id, span_id, timestamp, level, and domain-specific fields. Never use print() in production.
4. **REDACT SENSITIVE DATA** — Before any data reaches your logging pipeline, apply PII redaction to mask emails, phone numbers, SSNs, and full prompt contents.
5. **SHIP TO A BACKEND** — Send traces asynchronously to Langfuse (LLM-native) or OpenTelemetry (vendor-neutral). Overhead is typically under 5ms per request.

Instrumented Agent Loop

```
FUNCTION run_agent(user_message, trace_id):
  root_span = CREATE span("agent_loop", trace_id)

  WHILE agent has more steps:
    llm_span = CREATE child_span("llm.chat")
    response = CALL Claude API(message)
    LOG {trace_id, latency_ms, tokens_used}

    IF response needs tool call:
      tool_span = CREATE child_span("tool.run")
      result = EXECUTE tool(name, input)
      redacted = REDACT_PII(result)
      LOG {trace_id, tool_name, status}
    ELSE:
      final = REDACT_PII(response.text)
      BREAK

  SHIP trace to observability backend
  RETURN final
```

Tracing adds <5ms overhead per request. The performance insight you gain — finding a 3-second database bottleneck — far outweighs the microsecond cost of recording it.

Common Myths

"Observability is just logging with a fancier name."
Logging captures individual events as text lines. Observability combines **three pillars** — traces, logs, and metrics — into a system where you can answer questions you didn't anticipate when you wrote the code. Logs alone cannot show causal relationships between steps.

"Adding tracing will slow down my agent."
Modern tracing uses asynchronous batching — spans are queued in memory and flushed in background threads. Overhead is typically **under 5ms per request**. Your LLM calls take 200-3000ms; the tracing cost is invisible.

"I can just use print() and grep the output."
Works for one user locally. With 1,000 concurrent requests, print output interleaves randomly. Without **trace_id correlation**, you cannot reconstruct which log lines belong to which request.

Remember This

Observability transforms agent debugging from guessing to systematic investigation.

Capture **traces** (complete execution records), **spans** (timed units of work), and **structured logs** (machine-parseable JSON) for every request.

The performance gains from identifying bottlenecks through span timing typically pay for the infrastructure within days. If your trace shows LLM at 200ms and database at 3200ms, you instantly know where to optimize.

Test Yourself

1. What is the relationship between a trace and a span?

Answer: A trace is the top-level container representing the entire request lifecycle. **Spans are nested inside**, each representing a single unit of work (LLM call, tool invocation). Spans link via parent_span_id to form a tree.

2. Which of these should NEVER appear in production logs?

Answer: User email addresses and full prompt contents. PII violates privacy regulations; prompts may contain sensitive user data. Always hash user identifiers and log prompt summaries — not full text. Token counts, latency, and tool names are safe.

3. Trace: LLM (200ms) → tool.search_db (3200ms) → LLM (250ms). Where's the bottleneck?

Answer: The database tool call at 3200ms — 87.7% of total execution time. The LLM calls are fast. Add caching, optimize queries, or use connection pooling for the highest latency impact.

Monitoring & Continuous Improvement

Build dashboards, alerts, and feedback loops that keep your agent improving every week.

Track 6: Observability
Module 22 of 30

Monitoring & Continuous Improvement

Build dashboards, alerts, and feedback loops that keep your agent improving every week.

Agents Degrade Silently

After instrumenting agents with traces and logs (M19), you need to **act on that data**. This module teaches how to visualize problems through dashboards, alert the right people at the right severity, and establish repeatable improvement cycles.

LLM-based agents are fundamentally different from traditional APIs: model providers can change weights without your code changing, user patterns shift, and data sources degrade. Without continuous monitoring, **agent quality degrades silently**.

Teams that follow a systematic improvement cycle achieve **2-5 percentage points of eval score improvement per month** — a compounding gain that transforms a tolerated agent into one users love.

Elite Sports Teams Review Game Film

> A basketball team plays the next game hoping for the best. Win or lose, they just move on. No one reviews what worked, what broke, or what the opponent exploited. They plateau fast.

> Without systematic review, the same mistakes repeat. Defensive rotations keep failing. The coaching staff reacts to crisis losses but never prevents them. The team never compounds improvements.

> Elite teams **systematically review game film**: identify which plays worked, which failed, drill fixes, and measure improvement. AI agent continuous improvement follows the identical playbook — review failures, identify patterns, implement fixes, verify with evaluations, measure week over week.

The Improvement Cycle

1. **COLLECT SIGNALS** — Real-time user feedback (thumbs up/down), daily failure aggregation (pattern detection), and weekly eval dataset growth from production captures.
2. **MONITOR 4 METRICS** — Latency (p50/p95/p99), token usage and cost burn rate, success/failure rates by error type, and drift detection (output length, tool-call frequency trends).
3. **ROUTE ALERTS** — P1/Page for emergencies (>50% error rate). P2/Ticket for next-business-day (>10%). P3/Info for weekly review. Start conservative to avoid alert fatigue.
4. **DEPLOY SAFELY** — Start with 1-5% canary traffic. If metrics degrade, auto-rollback. Only expand to 25%, 50%, then 100% after canary passes.
5. **CLOSE THE LOOP WEEKLY** — Pull lowest-scoring 5% of responses, classify failures, add each as an eval test case, implement fixes, deploy via canary, measure improvement.

Metric Collection

```
FUNCTION run_with_metrics(message, trace_id):
  trace = CREATE trace(id=trace_id)
  start = CURRENT_TIME()

  TRY:
    response = CALL Claude API(message)
    latency = (CURRENT_TIME() - start) * 1000
    cost = CALCULATE(input_tokens, output_tokens)

    RECORD score(trace, "latency_ms", latency)
    RECORD score(trace, "tokens", total_tokens)
    RECORD score(trace, "cost_usd", cost)
    RECORD score(trace, "success", 1)

    RETURN response
  CATCH error:
    RECORD score(trace, "success", 0)
    RECORD score(trace, "error_type", error.name)
    RAISE error
```

Always wrap metric recording in a try/catch — don't crash the agent if monitoring is down. Serving users is more important than recording metrics.

Common Myths

"p50 latency is the only metric that matters."
p50 (median) shows typical experience, but **p95 and p99 reveal your most frustrated users**. A p50 of 1.2s with a p99 of 25s means 1 in 100 users waits 25 seconds. Those users are most likely to abandon your agent.

"We only need real-time feedback."
Real-time feedback signals something broke, not *why*. Daily aggregation reveals patterns. Weekly eval growth encodes fixes as permanent test cases. **Each speed answers different questions**: what broke, what keeps breaking, and did we actually fix it.

"More alerts = better monitoring."
Alert fatigue is the #1 killer of monitoring effectiveness. 50 noisy alerts that get ignored is worse than 5 well-tuned alerts that always get acted on. Start conservative; tighten gradually.

Remember This

Monitoring and continuous improvement are essential infrastructure, not optional polish.

Without dashboards you can't detect problems. Without tiered alerting you drown in noise or miss emergencies. Without feedback loops at multiple timescales, you fix crises but never prevent them.

The winning formula: **collect production signals → review failures weekly → grow eval datasets → deploy via canary**. Teams following this achieve compounding gains month by month.

Test Yourself

1. Error rate climbed from 2% to 8%. What alert severity?

Answer: P2/Ticket — create a ticket for next business day. Below the P1 threshold (50%) but significant enough to investigate. Should not wait for weekly review (P3).

2. Why do agent A/B tests need more samples than web A/B tests?

Answer: LLM outputs are non-deterministic. The same input produces different responses each time. This inherent variance widens confidence intervals, requiring **3-5x more samples** to achieve equivalent statistical confidence.

3. What are the four key metric categories for agent dashboards?

Answer: Latency (p50/p95/p99), **Token usage & cost** (burn rate), **Success/failure rates** (by error type), and **Drift detection** (output length, tool-call frequency, sentiment trends).

API Design & Deployment

Package your agent as a production API with streaming, containers, and cloud deployment.

Track 7: Production Deployment
Module 23 of 30

API Design & Deployment

Package your agent as a production API with streaming, containers, and cloud deployment.

From Script to Service

Your agent works locally. Now it needs to serve thousands of users 24/7. This module covers the journey from local script to production API: choosing the right **communication protocol**, **containerizing** with Docker, and **deploying** to cloud platforms with autoscaling.

The key insight: agent APIs are **I/O-bound, not CPU-bound**. An agent handler spends 95% of its time waiting for Claude API responses, database queries, and tool executions. This changes every scaling decision.

For most agents, **SSE (Server-Sent Events)** is the sweet spot — streaming tokens as they're generated, zero polling waste, over plain HTTP.

Letters, Phone Calls, and Radio

> You write a question, mail it, wait for the reply, repeat. Simple but slow. Every letter costs a stamp, even if the answer is "nothing new yet." If your friend is writing a long story, you keep mailing "are you done?" — wasteful polling.

> Both sides talk simultaneously in real time. But the line stays open, consuming resources even during silence. Overkill for most agent conversations.

> Your friend speaks into a microphone, you listen on a **one-way channel**. Tokens arrive as generated, zero polling waste. Connection closes cleanly when done. The sweet spot for agent APIs.

Script to Cloud in 5 Steps

1. **CHOOSE PROTOCOL** — REST for stateless ops, SSE for streaming agents (default), WebSocket only when clients must send data mid-stream.
2. **PICK MODEL HOST** — Direct Anthropic API (newest models first), AWS Bedrock (AWS IAM, billed via AWS), or Vertex AI (GCP IAM, billed via GCP). Same Claude, different doorway. Pick based on which cloud already holds your data and contracts.
3. **CONTAINERIZE** — Docker with multi-stage builds, -slim base images (150MB vs 900MB). Cache dependency layers, run as non-root, .dockerignore for secrets.
4. **DEPLOY TO CLOUD** — Cloud Run (SSE, 60-min timeouts, scale-to-zero), Lambda (short tasks <30s), or VMs (always-on high traffic). *Or skip the container entirely* — Bedrock AgentCore and Vertex Agent Engine host the agent loop for you.
5. **SCALE & PROTECT** — Autoscale on concurrent requests (not CPU). Per-user rate limiting. Circuit breakers for downstream failures.

Streaming SSE Endpoint

```
ENDPOINT POST /chat:
  IF NOT check_rate_limit(user_id):
    RESPOND 429 Too Many Requests

  CREATE SSE response stream
  SET headers: no-cache, keep-alive

  OPEN stream to Claude API:
    LOOP while events arrive:
      IF token event:
        SEND "event: token\ndata: {text}\n\n"
      IF tool_call event:
        SEND "event: tool_call\ndata: {name}\n\n"
      IF done event:
        SEND "event: done\ndata: {usage}\n\n"
        BREAK

  CATCH rate_limit_error:
    SEND "event: error" with retry_after

  CLOSE response stream
```

SSE rides on plain HTTP — no special infrastructure needed. Load balancers, CDNs, and proxies work out of the box. That's why SSE beats WebSocket for most agent APIs.

Common Myths

"Docker containers are like virtual machines."
A VM runs a full OS with its own kernel (gigabytes, minutes to boot). Containers share the host OS kernel — **start in milliseconds, use megabytes**. Think apartments vs. separate houses.

"Use the full base image for compatibility."
`python:3.12` is 900MB with compilers you'll never use. `python:3.12-slim` is **150MB** with everything needed. Smaller = faster deploys, faster scaling, fewer vulnerabilities.

"You always have to build and host the agent loop yourself."
You don't. **Bedrock AgentCore** and **Vertex AI Agent Engine** host a managed agent runtime — you upload a system prompt and tool definitions, they run the loop and own session state. No FastAPI server. Trade: less control over the loop, faster time-to-deploy.

Remember This

SSE + Docker + Cloud Run is the default stack — managed platforms are the escape hatch.

SSE streams tokens with zero polling waste. **Docker slim images** give portable, secure containers. **Cloud Run** provides SSE support, 60-min timeouts, and scale-to-zero billing.

Three independent decisions: **protocol** (REST/SSE/WebSocket), **model host** (direct Anthropic / Bedrock / Vertex), **compute platform** (Cloud Run / Lambda / VM). For each one, "skip it — let AWS or GCP run the agent loop" via Bedrock AgentCore or Vertex Agent Engine is a real fourth option.

The critical scaling insight: agents are **I/O-bound, not CPU-bound**. Always autoscale on concurrent requests, never on CPU.

Test Yourself

1. What is SSE's main advantage over REST polling?

Answer: SSE eliminates polling waste — the server pushes tokens as generated over a single HTTP connection. No repeated "are you done?" requests. Real-time delivery without constant polling overhead.

2. Why should API keys never go in Docker images?

Answer: Secrets persist in image layer history even if deleted later. Docker layers are immutable — anyone with image access can inspect all layers and extract the secret. Inject secrets via environment variables at runtime.

3. Agent task takes 45 seconds. Best platform?

Answer: Cloud Run. Native SSE streaming, up to 60-minute timeouts, scale-to-zero. Lambda's limited SSE support and shorter timeout make it less ideal for streaming agent responses.

Cost Optimization

Understand where every dollar goes and cut agent costs by 50-90% without sacrificing quality.

Track 7: Production Deployment
Module 24 of 30

Cost Optimization

Understand where every dollar goes and cut agent costs by 50-90% without sacrificing quality.

The Cheapest Token Is the One You Never Send

An agent call has **six cost components**: input tokens, output tokens (5x more expensive), tool executions, embeddings, compute overhead, and multi-turn multiplication. Most teams focus only on input tokens and are shocked when their bill is 10x higher.

The **hidden killer** is multi-turn multiplication: every turn resends the full conversation history, so costs grow geometrically. A 10-turn conversation costs roughly **55x** a single turn in cumulative input tokens.

Three strategies cut costs by 50-90%: **caching** (eliminate redundant calls), **model routing** (cheaper models for simple requests), and **token compression** (compress prompts, summarize history).

The Restaurant Bill

> You sit down expecting to pay for just the food (input tokens). A simple meal, straightforward bill.

> Six line items: the entrée (input tokens), dessert at **5x the price per ounce** (output tokens), tax (compute), tip for unfinished bottles (tool calls), appetizer sampler (embeddings), and a per-visit surcharge that **doubles every return visit** (multi-turn history).

> Know every line item. Order house wine instead of reserve (model routing). Share appetizers across tables (caching). Stop ordering the full menu when one course will do (token compression).

5 Steps to Cut Costs

1. **UNDERSTAND COST ANATOMY** — Input: $3/MTok. Output: $15/MTok (5x more!). History compounds geometrically per turn. Know which component dominates your bill.
2. **3-LAYER CACHING** — Prompt caching (90% savings, 5-min TTL that refreshes on hit). Response caching (semantic similarity). Embedding caching (Redis LRU for RAG).
3. **MODEL ROUTING** — Classify by complexity. Simple → Haiku ($0.25/MTok, 12x cheaper). Standard → Sonnet. Complex → Opus. 60-70% of traffic is Haiku-eligible.
4. **COMPRESS TOKENS** — System prompts (-63%). Rolling history summaries (-72%). max_tokens for output. Top-3 RAG chunks (-70%).
5. **INSTRUMENT TRACKING** — Log per-request cost: input tokens, output tokens, cache hits, model used, tool costs. Can't optimize what you don't measure.

Prompt Caching

```
SETUP:
  SYSTEM_PROMPT = create_block(
    text = "You are a support agent...",
    cache_control = "ephemeral"
  )

FUNCTION call_with_caching(user_msg):
  // First call: cache CREATED (full price)
  response1 = SEND to Claude(
    system = SYSTEM_PROMPT,
    messages = history + user_msg
  )

  // Second call within 5 min: cache HIT
  // System prompt costs only 10%
  response2 = SEND to Claude(
    system = SYSTEM_PROMPT,
    messages = history + new_msg
  )
  // Each hit RESETS the 5-min TTL
  // High-traffic: cache never expires
```

Prompt caching saves 90% on cached tokens. For agents handling >1 request per 5 minutes, the sliding TTL means the cache effectively never expires.

Common Myths

"Longer conversations are only slightly more expensive."
Dangerously false. Every turn resends full history — costs grow **geometrically**. A 10-turn conversation costs ~55x a single turn. Without history summarization, long conversations silently drain your budget.

"Input tokens are the main cost driver."
Output tokens cost **5x more** ($15 vs $3/MTok for Sonnet). An agent generating 500-token responses costs more in output than one sending 2,000 tokens of input. First optimization: can you make the output shorter?

"max_tokens limits how much I pay for input."
`max_tokens` only caps **output** length. Zero effect on input costs. To reduce input costs: prompt compression, history summarization, or caching.

Remember This

Three strategies can cut agent costs by 50-90% without quality loss.

Caching eliminates redundant API calls. **Model routing** sends 60-70% of traffic to Haiku at 12x lower cost. **Token compression** tackles the hidden cost multipliers.

The critical insight: **output tokens cost 5x more than input**, and **multi-turn history compounds geometrically**. Without measurement and mitigation, these become hidden drains that blow budgets.

Test Yourself

1. How do input vs. output token costs compare for Claude Sonnet?

Answer: Output tokens cost 5x more — $15/MTok vs $3/MTok for input. Controlling output length is one of the highest-leverage cost optimizations.

2. What is the TTL for Anthropic's prompt cache?

Answer: 5 minutes, refreshed on every hit. Sliding-window behavior means frequently-used prompts stay cached indefinitely. For agents with >1 req/5min, the cache effectively never expires.

3. Per-conversation cost doubles between turn 3 and turn 6. Why?

Answer: Multi-turn multiplication. Every turn resends full history. By turn 6, you send 6x the input of turn 1. Fix: sliding window with rolling summary — keep last 3 turns verbatim, summarize the rest.

Deploy Your Agent — Local, GCP & AWS

Wrap an agent as a REST API, containerize it with Docker, and deploy to cloud platforms — all with the same code.

Track 7: Production Deployment
Module 25 of 30

Deploy Your Agent — Local, GCP & AWS

Wrap an agent as a REST API, containerize it with Docker, and deploy to cloud platforms — all with the same code.

One Agent, Three Environments — Then Beyond

Deployment is a three-layer journey. First, you wrap your agent as a **REST API** so any client can talk to it over HTTP. Second, you package the API into a **Docker container** so it runs identically everywhere. Third, you push that container to a **cloud platform** so the world can reach it.

The critical insight: **your agent code never changes** between platforms. Only the infrastructure wrapper changes. The same agent runs on your laptop, on Google Cloud Run, and on AWS Lambda.

But that's only one of **four canonical topologies** you'll meet in production:

- **Sync API** — the lab pattern. Cloud Run / Lambda. Under 30 seconds.
- **Streaming chat** — SSE for copilots. Cloud Run + Redis sessions.
- **Async worker** — queue + workers for runs that take minutes-to-hours.
- **Scheduled batch** — cron + Anthropic Batches API (50% off).

The Brilliant Researcher

> Your agent is like a brilliant researcher who only works in person. You walk to their office, sit down, and ask your question face-to-face. Only one person at a time. No remote access.

> Nobody else can use this researcher. Your teammates cannot ask questions. Your web app cannot ask questions. The researcher is locked in one room with one person. And if you want to move offices, they refuse to come because the new building has a different layout.

> Wrapping the agent in a **REST API** gives the researcher a phone number — anyone can call. Putting them in a **Docker container** is like building a portable office that works in any building. Deploying to the **cloud** means cloning that portable office so hundreds of callers get their own researcher simultaneously.

The Deployment Pipeline

1. **WRAP AS API** — Create a REST server (e.g., FastAPI) with a `/query` endpoint and a `/health` check. The endpoint receives a question, passes it to the agent, and returns the response as JSON.
2. **CONTAINERIZE** — Write a Dockerfile that packages your OS, Python, dependencies, and code into one image. Use a non-root user and never bake secrets into the image.
3. **TEST LOCALLY** — Run the container with `docker run`, passing your API key at runtime via `-e` flag. Verify it responds to the same `curl` command.
4. **DEPLOY TO CLOUD** — Push the image to a registry (GCP Artifact Registry or AWS ECR). Deploy with resource limits, timeout settings, and max-instance caps to control costs.
5. **ADAPT FOR LAMBDA** — Add a thin adapter (like Mangum) that translates Lambda events into HTTP requests. Your FastAPI code stays unchanged — the adapter bridges the gap.

Agent Deployment Flow

```
// 1. Wrap agent as REST API
ENDPOINT POST /query(question):
  agent = CREATE Agent(api_key from ENV)
  result = agent.run(question)
  RETURN {answer: result}

ENDPOINT GET /health:
  RETURN {status: "healthy"}

// 2. Containerize
DOCKERFILE:
  FROM python-slim
  COPY code + dependencies
  RUN as non-root user
  EXPOSE port 8000
  // NEVER bake API keys here!

// 3. Deploy
LOCAL:  docker run -e API_KEY=xxx
GCP:    gcloud run deploy --max-instances=3
AWS:    sam deploy (Lambda + API Gateway)
```

The agent code is identical across all three. Only the outer wrapper changes: a server for Docker/Cloud Run, plus a thin adapter for Lambda.

Common Myths

"I can put my API key in the Dockerfile for convenience."
Docker images get pushed to registries and shared with teammates. Anyone who pulls the image can extract baked-in secrets. Always pass API keys at runtime via environment variables (`-e` flag) or a cloud secrets manager.

"Lambda is always cheaper than Cloud Run."
Lambda charges per invocation and per millisecond. For agents with 10-30 second response times, those milliseconds add up fast. Cloud Run charges per second with a generous free tier and scales to zero. For long-running agent tasks, Cloud Run is often cheaper.

"If my agent runs longer than 60 minutes, I'm stuck."
You're only stuck on the *sync* pattern. For long runs (deep research, multi-file refactors), switch to topology 3: HTTP returns a **job ID immediately**, a **queue** (SQS / Pub-Sub) hands the work to a **worker pool** (Fargate / GKE / Modal) with no timeout. The client polls or gets a webhook. Or skip building it: **Bedrock AgentCore** and **Vertex Agent Engine** host the loop and own session state for you.

Remember This

Pick the topology first — the platform follows.

The lab built **topology 1: sync API** — Docker, Cloud Run, Lambda. Same code, same agent, three deploys. Use this when requests finish in under 30 seconds.

Production agents fan out into three more shapes:

- **Streaming chat** → Cloud Run + SSE + Redis sessions
- **Async worker** → SQS / Pub-Sub + Fargate / GKE / Modal (no timeout)
- **Scheduled batch** → Cloud Scheduler + Anthropic Batches API (50% cheaper)

And when you'd rather not own any of it: **Bedrock AgentCore** or **Vertex Agent Engine** host the agent loop, persist sessions, and run tools — you just call `InvokeAgent`.

Test Yourself

1. Why should you NEVER bake API keys into a Docker image?

Answer: Images get pushed to registries and shared. Anyone who pulls the image can extract baked-in secrets. Always pass secrets at runtime via `-e` flags or a cloud secrets manager like GCP Secret Manager or AWS Secrets Manager.

2. Your agent takes 3 minutes per request. Cloud Run or Lambda?

Answer: Cloud Run. It supports timeouts up to 60 minutes, making it ideal for long-running agent tasks. Lambda maxes out at 15 minutes and is better suited for short, event-driven workloads like webhooks or scheduled triggers.

3. What does the Mangum library do in an AWS Lambda deployment?

Answer: It translates Lambda events into HTTP requests that FastAPI understands. Lambda sends events in its own format from API Gateway. Mangum converts them into standard ASGI requests, so you reuse 100% of your FastAPI server code without modification.

Capstone Project Series

Apply everything you have learned across five domain-anchored projects — from a single-tool assistant to a full production agent.

Track 8: Capstone Projects
Module 26 of 30

Capstone Project Series

Apply everything you have learned across five domain-anchored projects — from a single-tool assistant to a full production agent.

Five Projects, Three Domains, One Methodology

Capstone projects integrate every skill from the course into real, domain-specific agents. Five projects are organized into three tiers:

- **Tier 1** — Single-tool agent (2-3 hrs, beginner)
- **Tier 2** — RAG pipeline or multi-tool orchestration (4-5 hrs, intermediate)
- **Tier 3** — Multi-agent system with production deployment (6-8 hrs, advanced)

Every project follows a **five-phase methodology**: Requirements, Architecture, Build, Evaluate, Harden. Each phase maps to specific course modules so you always know what to apply.

Projects operate in three real-world domains: **Healthcare** (clinical pre-authorization), **B2B Ecommerce** (order tracking and fulfillment), and **Public Records** (UCC filings and lien risk). Same agent patterns, different constraints — that adaptability is the goal.

Building a House

> A capstone project is like building a house. Before picking paint colors and furniture (the fun parts), you need a foundation — a clear set of requirements: how many rooms, what is the lot size, what is the budget?

> Developers who jump straight to coding an agent build something that works for their first test case and collapses on the second. This "patch-and-pray" cycle wastes more time than planning ever would.

> The methodology maps directly: **foundation** (requirements) → **framing** (architecture) → **wiring** (tools and integrations) → **inspection** (evaluation) → **weatherproofing** (production hardening). Each phase has a checklist, and you do not move forward until the current phase is solid.

The Five-Phase Methodology

1. **REQUIREMENTS** — Read the domain brief. Define measurable success criteria ("returns correct status for 95% of valid inputs"). List edge cases. This prevents 80% of future bugs.
2. **ARCHITECTURE** — Choose the agent pattern that fits: single-turn tool-calling, ReAct loop, or multi-agent. Select tools, data sources, and state management.
3. **BUILD** — Build in vertical slices — one complete user flow at a time. Happy path first, then error handling, then edge cases.
4. **EVALUATE** — Run automated quality checks against your Phase 1 success criteria. Write eval test cases before you write agent code (test-driven development for agents).
5. **HARDEN** — Add guardrails, observability, and cost optimization. This is the difference between a demo and a deployable system.

Capstone Agent Architecture

```
DEFINE build_capstone(domain_brief):

  // Phase 1: Requirements
  requirements = PARSE(domain_brief)
  success_criteria = DEFINE_METRICS(requirements)
  eval_cases = GENERATE_TESTS(requirements)

  // Phase 2: Architecture
  pattern = SELECT_PATTERN(requirements)
    // single-tool | ReAct | multi-agent
  tools = REGISTER(domain_brief.tools)

  // Phase 3: Build (vertical slices)
  FOR each user_scenario in requirements:
    IMPLEMENT happy_path(scenario, tools)
    ADD error_handling(scenario)
    ADD edge_cases(scenario)
    RUN eval_cases  // test as you build

  // Phase 4-5: Evaluate + Harden
  score = RUN_FULL_EVAL(eval_cases)
  ADD guardrails, observability, caching
  RETURN production_ready_agent
```

The key insight: write your evaluation tests in Phase 1 (before any code), then run them continuously as you build each vertical slice.

Common Myths

"I should start with the hardest capstone to learn the most."
Capstone 5 assumes you can already build single-tool agents, RAG pipelines, and multi-agent systems. Skipping building blocks means you spend all your time debugging fundamentals instead of learning production deployment.

"I need to build all five capstones."
One well-executed capstone with a thorough self-assessment teaches more than five rushed ones. Quality over quantity — aim for a self-assessment score of 20+/25 on one project before moving to the next.

"Architecture design is overkill for a single-tool agent."
Even Capstone 1 benefits from 10 minutes of planning. Students who skip this step consistently produce worse code because they make structural decisions mid-implementation that create messy, hard-to-test code.

Remember This

Methodology beats talent.

The five-phase approach (Requirements, Architecture, Build, Evaluate, Harden) is the difference between agents that work on test day and agents that work in production.

- **73% of production agent failures** trace back to skipping requirements or architecture
- **Three domains** (Healthcare, Ecommerce, Public Records) force you to adapt core skills to different constraints
- **Write eval tests first** — before any agent code — so you build toward success criteria instead of discovering failures after the fact

Pick the tier that matches your completed tracks, follow the methodology, and focus on depth over breadth. One thoroughly built capstone is worth more than five half-finished ones.

Test Yourself

1. What are the five phases of the capstone methodology, and which phase prevents the most bugs?

Answer: Requirements, Architecture, Build, Evaluate, Harden. Phase 1 (Requirements) prevents 80% of future bugs by defining success criteria and edge cases before any code is written. Teams that skip it end up in a "patch-and-pray" cycle.

2. Why does the capstone series use three different industry domains instead of one?

Answer: To force adaptability. The same core agent skills (tool use, RAG, guardrails) must adapt to different constraints: clinical compliance in Healthcare, real-time multi-API coordination in Ecommerce, and messy data with entity resolution in Public Records. A developer who can only build agents in one domain has limited value.

3. When should you write your evaluation test cases — before or after building the agent?

Answer: Before — in Phase 1 (Requirements). This is test-driven development applied to agents. When you define what "success" looks like upfront, you build toward it. Writing tests after building means you discover failures after the fact instead of preventing them.

What's Next — The Agent Frontier

Explore emerging agent patterns — agent-to-agent protocols, computer use, extended thinking, and your 90-day development roadmap.

Track 8: Capstone Projects
Module 27 of 30

What's Next — The Agent Frontier

Explore emerging agent patterns — agent-to-agent protocols, computer use, extended thinking, and your 90-day development roadmap.

The Agent Ecosystem Is Forming

You have built agents that reason, retrieve, coordinate, and deploy. The frontier is about what happens when agents stop working alone and start working **together**.

Four emerging patterns are reshaping what agents can do:

- **Agent-to-agent protocols** — standardized ways for agents to discover each other, negotiate capabilities, and delegate tasks
- **Computer use** — agents that see screens and operate any software, not just APIs
- **Extended thinking** — dedicated reasoning compute that dramatically improves complex planning
- **Larger context windows** — process entire codebases or regulatory documents in a single request

Each capability does not just add a feature — it **unlocks entire categories** of applications that were previously impossible.

The Smartphone Moment

> Early smartphones could make calls and send texts. Then came cameras, GPS, and app stores. Each new capability did not just add a feature — it unlocked entire application categories nobody had imagined. Before phone cameras, you could not photograph a whiteboard and share it instantly. You did not miss it because you could not imagine it.

> Today most agents are standalone bots. They do one thing well but cannot discover or collaborate with other agents. Connecting Claude to a new data source requires custom integration code. Companies report spending **40–60% of development time** on integrations alone.

> Agent-to-agent protocols are the **"API moment" for AI** — the universal plug that lets any agent discover, negotiate with, and delegate to any other agent. Computer use is the "camera chip" — one new capability that unlocks automation of every legacy system with a GUI. Extended thinking is the "app store" — dedicated reasoning compute that enables complex planning no prompting trick could achieve.

Four Frontier Patterns

1. **AGENT DELEGATION** — An orchestrator agent reads a specialist's capability manifest (a machine-readable "resume" in JSON), matches the task to the right specialist, sends structured input, and receives typed output. Like microservices for AI.
2. **COMPUTER USE** — Claude receives a screenshot, decides where to click or type, then receives the next screenshot. Slower and costlier than tool use, but works with any software that has a visual interface — including 15-year-old legacy systems with no API.
3. **EXTENDED THINKING** — A dedicated reasoning phase before Claude responds. Not the same as "think step by step" prompting — this allocates separate compute for reasoning. Thinking tokens are visible for debugging and billed separately.
4. **AGENT MARKETPLACES** — Directories where developers publish agents with standardized capability descriptions. An orchestrator can search at runtime, discover a specialist it has never seen, verify its manifest, and delegate — like npm for agents.

Agent Discovery and Delegation

```
DEFINE handle_complex_task(task):

  // Step 1: Find the right specialist
  specialists = SEARCH marketplace
    WHERE capabilities MATCH task.domain

  // Step 2: Read capability manifest
  manifest = GET specialist.manifest
  IF manifest.constraints NOT MET:
    RETURN fallback_to_general_agent()

  // Step 3: Delegate with structured I/O
  result = DELEGATE task TO specialist
    WITH input = task.structured_data
    WITH timeout = 30 seconds

  // Step 4: Validate and return
  IF result.confidence > 0.85:
    RETURN result
  ELSE:
    ESCALATE to human reviewer
```

The orchestrator discovers specialists at runtime, verifies their capabilities via manifests, delegates with structured data, and validates confidence before returning results.

Common Myths

"Agents talk to each other in natural language."
No. Agent-to-agent communication uses structured data — JSON schemas, typed inputs and outputs, capability manifests. Natural language is for humans. Machine-to-machine communication needs precision, not prose.

"Agent marketplaces will replace custom development."
Marketplaces provide general-purpose specialists, but your business logic, domain rules, and proprietary data still require custom agents. Think of it like npm: you use packages for common tasks, but you still write your own application code.

"Extended thinking is just chain-of-thought prompting."
Chain-of-thought is a prompting trick — you ask the model to show its work using the same compute. Extended thinking is model-native: Claude allocates **dedicated compute** specifically for reasoning before writing the response, producing dramatically better results on complex tasks.

Remember This

Each new capability unlocks new agent categories.

The agent frontier is defined by four converging trends:

- **Agent protocols** turn isolated bots into a collaborative ecosystem — reducing integration time from 40–60% of development to near zero
- **Computer use** lets agents operate any software with a GUI, unlocking automation of legacy systems
- **Extended thinking** enables complex planning and architecture decisions that no prompting trick can achieve

Your 90-day roadmap: **Month 1** — build and deploy one production agent. **Month 2** — add multi-agent coordination and observability. **Month 3** — integrate emerging capabilities (computer use, extended thinking) and publish to an agent marketplace.

Test Yourself

1. How do agents discover each other's capabilities in an agent-to-agent protocol?

Answer: Through capability manifests — structured JSON descriptions of what an agent can do, what inputs it accepts, what outputs it produces, and what constraints it has. The orchestrator reads the manifest to decide whether to delegate a task to that specialist.

2. How does computer use differ from tool use?

Answer: Tool use calls structured functions you define; computer use operates visual interfaces like a human. Claude receives screenshots and issues mouse/keyboard actions. It is slower, less predictable, and more expensive — but works with any software that has a GUI, including legacy systems with no API.

3. Why is extended thinking different from chain-of-thought prompting?

Answer: Extended thinking allocates dedicated compute for reasoning before generating the response. Chain-of-thought is a prompting trick that uses the same compute. Extended thinking produces thinking tokens (visible for debugging, billed separately) and dramatically improves performance on complex multi-step tasks.

Claude Code Mastery

Master CLAUDE.md configuration, slash commands, skills, plan mode, and CI/CD integration — the core of Domain 3 on the cert exam.

Track 9: Certification Prep
Module 28 of 30

Claude Code Mastery

Master CLAUDE.md configuration, slash commands, skills, plan mode, and CI/CD integration — the core of Domain 3 on the cert exam.

Claude Code Is a Configurable Development Environment

Claude Code is not just a chatbot in your terminal. It is a **fully configurable development environment** with layered configuration, reusable workflows, and CI/CD integration.

The power comes from three systems working together:

- **CLAUDE.md hierarchy** — cascading configuration files (user, project, directory) that tell Claude how to behave in each context
- **Commands and skills** — reusable workflows you trigger with slash commands, with optional context isolation so exploration noise stays out of your main session
- **Plan mode and built-in tools** — choosing between planning first vs acting immediately, using tools like Read, Edit, Bash, Grep, and Glob

This module covers **Domain 3 of the Claude Certified Architect exam** (~20% of the exam), so every concept here is directly testable.

The CSS Cascade

> CLAUDE.md files work like CSS cascading stylesheets. Think of three layers: a global stylesheet (body tag defaults), a page-level class (section overrides), and an inline style (element-specific). The most specific rule always wins.

> Without cascading, every developer would copy-paste the same rules into every project and directory. Personal preferences (like tab width) would conflict with team standards. Changes would mean updating dozens of files.

> **User-level** (~/.claude/CLAUDE.md) is your personal global stylesheet. **Project-level** (.claude/CLAUDE.md) is the team's shared rules, committed to git. **Directory-level** (src/api/CLAUDE.md) is the inline override for specific code areas. More specific always wins, and all levels merge together.

Four Key Systems

1. **CLAUDE.md CASCADE** — Three config levels merge together. User-level sets personal preferences (indentation, commit style). Project-level sets team standards (tech stack, API patterns). Directory-level adds path-specific rules (validation schemas for API code, parameterized queries for DB code). Most specific wins conflicts.

2. **COMMANDS vs SKILLS** — Commands are markdown files in .claude/commands/ that become slash commands. Skills live in .claude/skills/ and add two extras: *context: fork* runs in isolation (exploration noise stays out of your session), and *automatic invocation* (Claude triggers the skill when your request matches its description).

3. **PLAN MODE** — Toggle with Shift+Tab. In plan mode, Claude analyzes and outlines a strategy before executing. Use it for complex, risky, or unfamiliar tasks. Skip it for simple, well-understood changes where planning adds overhead without value.

4. **CI/CD INTEGRATION** — Run Claude Code in non-interactive mode within pipelines. Use separate sessions (--session-id) so each CI job gets an unbiased review. Combine with the Batch API for high-volume processing at 50% cost discount.

Claude Code Workflow

```
WHEN Claude Code starts:
  config = LOAD ~/.claude/CLAUDE.md
  config = MERGE(config, .claude/CLAUDE.md)
  config = MERGE(config, {dir}/CLAUDE.md)
  // most specific level wins conflicts

WHEN user types /command-name:
  prompt = READ .claude/commands/{name}.md
  tools = frontmatter.allowed-tools
  EXECUTE prompt WITH tools

WHEN user triggers a skill:
  IF skill.context == "fork":
    child = NEW_CONTEXT(isolated=true)
    result = child.EXECUTE(skill.prompt)
    RETURN result  // only summary
  ELSE:
    EXECUTE skill.prompt  // in main ctx

WHEN plan mode is ON (Shift+Tab):
  plan = ANALYZE(task, risks, steps)
  SHOW plan TO user
  IF user approves: EXECUTE plan
```

Configuration cascades at startup, commands and skills define reusable workflows, and plan mode gates execution behind analysis.

Common Myths

"Put all your preferences in the project-level CLAUDE.md."
Personal preferences (indentation, commit style, editor behavior) belong in user-level ~/.claude/CLAUDE.md. Project-level is for team standards (tech stack, API patterns, database rules). Mixing them forces your personal choices on the whole team. The cert exam tests this distinction.

"Commands and skills are the same thing."
Skills add two critical capabilities commands lack: *context: fork* for isolated execution (exploration noise stays out of your session), and automatic invocation based on description matching. Use commands for simple tasks; use skills when you need isolation or auto-triggering.

"Always use plan mode — planning is always better."
Plan mode adds overhead. For simple, well-understood tasks (rename a variable, fix a typo), direct execution is faster and equally safe. Reserve plan mode for complex, risky, or unfamiliar tasks where the cost of mistakes justifies the planning time.

Remember This

Configure, automate, isolate.

Claude Code mastery comes from three practices:

- **Configure at the right level** — personal preferences in user-level, team standards in project-level, path-specific rules in directory-level
- **Automate with commands and skills** — make reusable workflows instead of repeating prompts; use skills with context: fork for exploration-heavy tasks
- **Choose your execution mode** — plan mode for complex/risky tasks, direct execution for simple changes, CI/CD integration for automated pipelines

This covers Domain 3 of the cert exam (~20%). The key anti-patterns: putting personal prefs in project config, using commands instead of skills for heavy exploration, and always/never using plan mode instead of matching it to task complexity.

Test Yourself

1. Where should personal editor preferences (like indentation style) be configured in Claude Code?

Answer: In user-level ~/.claude/CLAUDE.md. Personal preferences apply to you across all projects. The project-level .claude/CLAUDE.md is for team standards shared via git. Putting personal prefs at the project level forces your choices on teammates — a key anti-pattern on the cert exam.

2. When should you use a skill with context: fork instead of a regular command?

Answer: When the task involves reading many files or heavy exploration. A command dumps all intermediate work (file contents, search results) into your main context window. A skill with context: fork runs in an isolated context — only the final summary returns to your session, keeping it clean and focused.

3. What happens when a directory-level CLAUDE.md conflicts with the project-level CLAUDE.md?

Answer: The directory-level rule wins. CLAUDE.md uses a cascading merge — all levels combine, but more-specific levels override more-general ones on conflicts. Directory-level is the most specific, then project-level, then user-level. Non-conflicting rules from all levels still apply.

Hooks, Sessions & Agent SDK

Master programmatic enforcement with hooks, session persistence, and the Agent SDK for building production agents.

Track 9: Certification Prep
Module 29 of 30

Hooks, Sessions & Agent SDK

Master programmatic enforcement with hooks, session persistence, and the Agent SDK for building production agents.

Three Layers of Agent Control

Production agents need three capabilities beyond the basic Messages API: an **SDK** that manages the agentic loop automatically, **hooks** that enforce business rules deterministically, and **sessions** that persist state across interactions.

The **Agent SDK** replaces 40+ lines of manual loop code with a single `query()` call. It handles tool dispatch, `stop_reason` checking, and message history internally.

Hooks are code that runs before or after every tool call — outside the model's control. Unlike prompt instructions (which are probabilistic), hooks enforce rules 100% of the time. A prompt saying "never refund over $500" can be ignored. A hook that blocks refunds over $500 cannot.

Sessions let you name, resume, and fork conversations. Forking creates parallel branches for exploration without polluting the main session.

Together these cover **Domain 1 of the Claude Certified Architect exam** (~25% of the exam weight).

The Phone Call vs. The Contractor

> Using the Messages API is like calling someone on the phone. You make a call, they respond, you hang up. For a multi-step task, you manage the entire flow yourself: call, listen, respond, call again, check if they are done, handle errors if the line drops. Every detail is your responsibility — dozens of lines of identical boilerplate code.

> Every agent developer writes the same while loop, the same `stop_reason` check, the same tool dispatch logic. Meanwhile, critical business rules live in prompts that the model can ignore. One missed refund limit check means a real financial loss and a compliance violation.

> The Agent SDK is like hiring a contractor: you describe the job and hand over the tools, and the contractor manages the process. Hooks are the lock on the liquor cabinet — deterministic code that runs every time, regardless of what the model "wants" to do. Sessions are your saved bookmarks so you can close the terminal and pick up tomorrow.

SDK, Hooks & Sessions

1. **AGENT SDK** — The `query()` function runs the full Reason-Act-Observe loop. It checks `stop_reason` on every response: `tool_use` means continue (execute the tool, send result back), `end_turn` means stop. The `maxTurns` parameter is a safety net, NOT the primary stopping mechanism.

2. **HOOKS** — Configured in `.claude/settings.json`. **PreToolUse** hooks run before a tool call (can allow, block, or modify parameters). **PostToolUse** hooks run after (can pass, block, or transform results). They execute as external scripts — the model cannot bypass them.

3. **SESSIONS** — `claude --session my-task` names a session. `claude --resume` picks up where you left off. `/fork` creates a branched copy for parallel exploration. Forks inherit history up to the branch point, then diverge independently.

4. **SUBAGENTS** — The Task tool spawns subagents with isolated context windows. A coordinator breaks complex work into specialist subtasks. Each subagent sees only the context you explicitly pass — not the coordinator's full history. This isolation prevents cross-contamination.

Hook + SDK + Subagent Pattern

```
CONFIGURE hooks in settings.json:
  PostToolUse "issue_refund":
    IF input.amount > 500:
      BLOCK "Exceeds limit. Escalated."
    ELSE: ALLOW

DEFINE run_agent(task):
  // SDK manages the agentic loop
  FOR message IN query(task, tools, maxTurns=15):
    IF message.type == "assistant":
      PROCESS message.content

DEFINE coordinate(complex_request):
  subtasks = DECOMPOSE(complex_request)
  FOR EACH subtask:
    // Task tool spawns isolated subagent
    result = TASK(prompt=subtask + context)
  RETURN AGGREGATE(results)
```

The hook enforces the refund limit as deterministic code. The SDK handles the agentic loop with one function call. The coordinator delegates to subagents via the Task tool, passing context explicitly because subagents have isolated context windows.

Common Myths

"Prompt instructions are enough for critical business rules."
Prompts are probabilistic — "never refund over $500" works 99% of the time, but the 1% failure creates real financial loss and compliance violations. Hooks run as external code, deterministically, every single time. Use prompts for style and tone; use hooks for anything that MUST be enforced.

"maxTurns controls when the agent stops."
`maxTurns` is a safety net to prevent infinite loops, NOT the primary stopping mechanism. The agent terminates naturally via `stop_reason: 'end_turn'`. If `maxTurns` fires, something went wrong — the agent could not complete the task in the expected number of steps.

"Subagents can see the coordinator's conversation."
Subagents spawned via the Task tool have completely isolated context windows. They see ONLY the prompt you pass them. If the user mentioned "Acme Corp" three messages ago in the coordinator, the subagent does not know unless you include it. Forgetting to pass context is the most common multi-agent bug.

Remember This

Prompts suggest. Hooks enforce. The SDK automates.

Three rules for the certification exam and production systems:

- **Agent SDK** — use `query()` instead of manual while loops. Terminate on `stop_reason`, never by parsing text like "I'm done"
- **Hooks** — PreToolUse and PostToolUse hooks are deterministic code. Use them for refund limits, compliance checks, PII redaction — anything that cannot tolerate even 1% failure
- **Sessions** — use `fork_session` for parallel exploration (shared history), use the Task tool for delegation (isolated context)

This module covers Domain 1 of the Claude Certified Architect exam — approximately **25% of the total exam weight**.

Test Yourself

1. What does stop_reason: 'tool_use' mean in a Claude API response?

Answer: Claude wants to call a tool — continue the agentic loop. Execute the requested tool, send the result back to Claude, and let it continue. The loop only stops when `stop_reason` is `'end_turn'` (task complete) or `'max_tokens'` (output buffer full).

2. Why should you use hooks instead of prompt instructions for a $500 refund limit?

Answer: Hooks are deterministic code; prompts are probabilistic suggestions. A prompt saying "never refund over $500" can be ignored by the model. A PostToolUse hook that checks the refund amount runs as an external script every time — the model cannot bypass, override, or be "convinced" to skip it.

3. What is the difference between fork_session and the Task tool for parallel work?

Answer: fork_session copies full history; Task tool creates isolated context. Use `fork_session` when a single user wants to explore two approaches with shared history. Use the Task tool when delegating to specialist subagents that should only see the context you explicitly pass — not the coordinator's entire conversation.

Certification Exam Prep

Anti-patterns, scenario walkthroughs, and mock exam strategy for the Claude Certified Architect — Foundations exam.

Track 9: Certification Prep
Module 30 of 30

Certification Exam Prep

Anti-patterns, scenario walkthroughs, and mock exam strategy for the Claude Certified Architect — Foundations exam.

Spot the Anti-Patterns

The Claude Certified Architect exam covers **5 domains**: Agentic Architecture, Tool Design & MCP, Claude Code Config, Prompt Engineering, and Context & Reliability. Passing score is **720/1000**.

Every wrong answer on the exam is a real anti-pattern that developers actually ship to production. The exam doesn't test trivia — it tests whether you can **spot plausible-but-broken approaches** under pressure.

There are **18 cataloged anti-patterns** across all 5 domains and **6 exam scenarios** (support agent, code gen, multi-agent research, dev productivity, CI/CD, data extraction). Each scenario tests multiple domains at once.

The key insight: roughly **40% of questions** test architectural judgment — design decisions, not API calls. Knowing the SDK isn't enough.

Medical Boards

> Before medical boards tested clinical knowledge, doctors learned from textbooks alone. Knowing the right diagnosis isn't enough — you also need to recognize the wrong diagnoses that look plausible. A patient with chest pain could be having a heart attack, acid reflux, or a pulled muscle.

> The textbook teaches the right answer, but the boards test whether you can spot the impostors. Under time pressure, plausible-sounding wrong answers trip up even experienced practitioners.

> The certification exam works the same way. Each question has **one correct pattern and three anti-patterns**. The anti-patterns aren't random nonsense — they're approaches that seem reasonable but fail in production. Knowing WHY each fails is more valuable than memorizing the correct answer.

Exam Strategy in 5 Steps

1. **READ THE SCENARIO** — Each question belongs to one of 6 scenarios (support agent, code gen, multi-agent, dev productivity, CI/CD, data extraction). Identify which scenario and which domain is being tested.
2. **ELIMINATE ANTI-PATTERNS** — Of 4 answer choices, 3 are anti-patterns. Look for red flags: prompt-based enforcement of business rules, self-reported confidence, generic error messages, parsing natural language for control flow.
3. **APPLY THE DETERMINISM RULE** — The exam consistently rewards deterministic solutions over probabilistic ones. Hooks over prompt instructions. Programmatic checks over model judgment. Structured errors over generic strings.
4. **CHECK THE VALIDATION-RETRY PATTERN** — Domain 4's most-tested pattern: tool_use guarantees JSON structure but NOT value correctness. Always validate, give specific field-level error feedback, and cap retries at 3 per field.
5. **VERIFY PROVENANCE** — For multi-agent questions, the correct answer always tracks source, confidence, timestamp, and agent ID. Resolution priority: official source > aggregator > user-submitted. Freshest data breaks ties.

Exam Question Decision Tree

```
DEFINE answer_exam_question(question):

  scenario = IDENTIFY_SCENARIO(question)
  domain = IDENTIFY_DOMAIN(question)

  // Step 1: Eliminate anti-patterns
  FOR each option IN answers:
    IF option uses prompt-based enforcement:
      ELIMINATE  // Anti-pattern #3
    IF option trusts self-reported confidence:
      ELIMINATE  // Anti-pattern #5
    IF option returns generic errors:
      ELIMINATE  // Anti-pattern #6

  // Step 2: Prefer deterministic solutions
  remaining = non-eliminated options
  PREFER option with hooks over prompts
  PREFER option with programmatic checks
  PREFER option with structured errors

  RETURN highest-determinism option
```

This decision tree works for most exam questions: identify the scenario, eliminate known anti-patterns, then pick the most deterministic remaining option.

Common Myths

"I just need to memorize the 18 anti-patterns."
Memorization alone won't work. The exam presents novel scenarios where you need to APPLY the principles. You might recognize anti-pattern #3 (prompt-based enforcement) in isolation, but can you spot it embedded in a CI/CD question about code review guardrails? The exam tests application, not recall.

"If I know the Anthropic API well, I'll pass."
API knowledge is necessary but insufficient. Roughly 40% of questions test architectural judgment — when to use multi-agent vs single-agent, when to escalate, how to handle conflicting data. These are design decisions, not API calls.

"Two answers often look correct — just pick either."
Both "use a hook" and "add a system prompt instruction" technically enforce a refund limit. The difference: hooks are deterministic and can't be bypassed by prompt injection. The exam rewards understanding WHY one approach is better, not just that it works.

Remember This

Determinism wins every exam question.

The single principle that unlocks most exam questions:

- **Hooks over prompts** — programmatic enforcement can't be bypassed by prompt injection
- **stop_reason over NL parsing** — check the API field, don't parse "I'm done"
- **Structured errors over generic** — `{isError, category, isRetryable}` not "failed"
- **Specific feedback over "try again"** — field-level errors boost self-correction from 30% to 80%
- **Provenance over trust** — track source, confidence, timestamp for every claim

Know the 6 scenarios, eliminate anti-patterns first, and always prefer the deterministic option. Passing score is 720/1000 — aim for 750+.

Test Yourself

1. A support agent must enforce a $500 refund limit. Which approach does the exam reward?

Answer: Use a programmatic hook to intercept process_refund calls. Hooks are deterministic and can't be bypassed by prompt injection. Adding "Never approve refunds over $500" to the system prompt is anti-pattern #3 — prompt-based enforcement of business rules.

2. tool_use returns valid JSON for a medical extraction. Is the data guaranteed correct?

Answer: No. tool_use guarantees structural correctness (valid JSON matching the schema), but NOT semantic correctness. A CPT code field containing "knee surgery" is valid JSON but not a valid CPT code. Always validate business rules after extraction, and provide specific field-level error feedback for retries.

3. Two subagents return conflicting entity data. How should the coordinator resolve it?

Answer: Use provenance metadata with a deterministic priority chain. Trust official sources over aggregators. If equal reliability, trust fresher data. If still conflicting, flag as "contested" and route to human review. Never delegate source reliability ranking to the model — hardcode it in resolution logic.

MODULE

Cert Domain 5.5+5.6 Deep Dive

Track 9: Cert Prep

MODULE M27B · 32 of 32 · Final

Cert Domain 5.5+5.6 Deep Dive

Provenance · Temporal · Stratified Review · Synthesis · 12 min read

The Big Idea

Domains 5.5 and 5.6 are the **biggest single gap** in most exam prep — and the cert tests them as one unified discipline.

Every claim a knowledge agent emits must be tagged with **provenance** (source), tagged with **time** (valid_from / valid_to), confidence-scored at the **field level** (not document), and synthesized to distinguish **well-established** from **contested** claims.

Without these four pillars, your output may be right — but not exam-compliant.

The Analogy

> A Meta-Analysis Paper
> "Result X is replicated across 7 studies (well-established)" reads very differently from "Result Y is supported by 3 studies and contradicted by 2 (contested)."
>
> A reader can act on the first. The second requires more digging — and silently picking one would be malpractice.
>
> A good synthesis layer doesn't collapse them. It surfaces both with their source pointers and lets the caller decide.

Sources Agree

3 chunks
↓
well_established
↓
all source IDs

Sources Disagree

2 chunks
↓
contested
↓
BOTH source IDs

Single Source

1 chunk
↓
single_source
↓
needs_review

How It Works — The Four Pillars

1

PROVENANCE — Every claim returns as `{claim, source_id, confidence}`. The schema enforces it — not the prose. Parenthetical citations don't count.

2

TEMPORAL — Store `{value, valid_from, valid_to, source}`. Query with explicit as-of predicates. `valid_to IS NULL` = current. Most temporal bugs come from omitting that filter.

3

FIELD-LEVEL CONFIDENCE — A document at 88% overall might have one critical field at 30%. Aggregate hides it. Score per field, not per document.

4

STRATIFIED HUMAN REVIEW — Sample N from *each* confidence bucket (high / med / low). Uniform sampling under-reviews low-conf. Top-N over-reviews easy ones. Stratified surfaces defects across grades.

5

SYNTHESIS OUTPUT — Schema must support `claim_status: established | contested | single_source`. When sources disagree, surface BOTH source pointers. Never silently pick one.

The Pseudocode

The `ProvenancedSynthesizer` pattern.

```
// 1. EXTRACT — per-chunk claim + confidence
FOR each retrieved chunk:
  claim = EXTRACT(chunk)
  conf  = SCORE(claim, chunk)
  store {claim, source: chunk.id,
         confidence: conf,
         valid_from, valid_to}

// 2. CLUSTER by entailment
clusters = GROUP claims by entailment
FOR each cluster:
  IF all sources agree:
    OUT.well_established += {
      claim, sources[], confidence }
  ELSE IF claims contradict:
    OUT.contested += {
      claim_a, sources_a,
      claim_b, sources_b }
  ELSE:
    OUT.single_source += {
      claim, source, needs_review:true }

// 3. FLAG temporal staleness
FOR each claim:
  IF claim.valid_to < query.as_of:
    OUT.temporal_warnings += {
      claim, freshness:"stale" }

// 4. STRATIFIED review sample
FOR each bucket in OUT:
  pick N for human review
```

Common Misconceptions

"Prose with parenthetical citations is fine"

No. The cert tests structured `{claim, source_id, confidence}`. Citations in narrative pass humans but fail compliance audits and can't be programmatically retracted when a source is invalidated.

"When sources agree, just collapse them into one"

No. Preserve all source pointers. "Well-established" is itself an output property — multiple sources are the evidence. Collapsing makes provenance unrecoverable.

"Temporal data = a timestamp on the row"

No. Temporal data is `{value, valid_from, valid_to}`. Missing the `valid_to` filter returns every historical version. "Acme's CEO is Bob" reported when it's been Alice for 6 months is the classic bug.

Key Takeaway

It's a schema problem, not a prompting problem

Domain 5.5/5.6 is the **biggest gap** in most prep — and the fix isn't a better prompt. It's a stricter output schema.

Every claim must be: **provenance-tagged**, **time-aware**, **field-confident**, and **synthesis-classified**.

Memorize the trap: **contested claims must include BOTH source pointers** (`sources_a` AND `sources_b`). Silent disagreement resolution is the most common cert exam failure mode.

Quick Quiz

Tap each card to reveal the answer. These are exam-style traps.

Q1: Two payer documents disagree on CPT coverage. The agent picks one and reports it. What's wrong?
Tap to reveal

Answer: Silent disagreement resolution. The output should be a CONTESTED claim with BOTH source pointers (`sources_a` + `sources_b`). Never silently pick.

Q2: Document-level confidence is 88%. Is per-field confidence still needed?
Tap to reveal

Answer: Yes. A document at 88% can have one critical field at 30%. Aggregate confidence hides field-level risk — especially for high-stakes extraction. Field-level surfaces what aggregates bury.

Q3: Which sampling strategy aligns with cert recommendations for human review?
Tap to reveal

Answer: Stratified. N from each confidence bucket. Uniform under-reviews low-conf cases. Top-N over-reviews easy ones. Stratified surfaces defects across all grades evenly.